

PAPAMA · PHASE 1

# Technical Administration Guide

---

|              |                                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------------|
| Version      | 1.2 (Phase 1)                                                                                           |
| Last updated | 29 September 2026                                                                                       |
| Supersedes   | Technical Administration Guide v1.1 (August 2026)                                                       |
| Audience     | Administrators, compliance officers, Food Partner managers, implementation partners and technical teams |

---

Section 3 is the operating manual for administrators. Section 9 is the configuration reference. Section 10 answers the questions that come up most often. Everything marked **Built** exists in the running platform; everything marked **Planned** does not yet.

# Contents

|                                             |           |
|---------------------------------------------|-----------|
| <b>How to Use This Guide</b>                | <b>6</b>  |
| Terminology                                 | 6         |
| Conventions                                 | 6         |
| What Changed in Version 1.2                 | 6         |
| <b>1. What is pApAmA?</b>                   | <b>8</b>  |
| How pApAmA Works                            | 8         |
| Founding Principles                         | 8         |
| Platform Roles                              | 8         |
| <b>2. How to Sign In</b>                    | <b>10</b> |
| Role Purposes                               | 10        |
| Login Pages                                 | 10        |
| Registration and Approval Flow              | 10        |
| Forgot Password                             | 11        |
| <b>Go-Live Checklist</b>                    | <b>12</b> |
| Business Configuration (Admin Console)      | 12        |
| Technical Environment Variables             | 13        |
| <b>3. For Admins — Running the Platform</b> | <b>14</b> |
| 3.1 Admin Dashboard (Home)                  | 14        |
| 3.2 Managing Donations                      | 14        |
| 3.3 Managing Tokens                         | 15        |
| 3.4 Beneficiary Registration Approvals      | 18        |
| 3.5 Managing Food Partners                  | 18        |
| 3.6 Menu Approvals                          | 20        |
| 3.7 Reviewing Meal Proofs                   | 20        |
| 3.8 Settlements and Food Partner Payouts    | 21        |
| 3.9 Volunteers and Token Allocation         | 23        |
| 3.10 Fraud Monitoring                       | 23        |
| 3.11 Emergency Mode                         | 24        |
| 3.12 Analytics and Reports                  | 26        |
| 3.13 System Configuration                   | 27        |
| 3.14 Other Admin Sections                   | 28        |
| 3.15 Contribution Report                    | 28        |
| 3.16 Exception Queue                        | 29        |
| 3.17 Audit Selections                       | 29        |
| 3.18 Emergency Register                     | 29        |
| 3.19 Offline Transaction Register           | 30        |
| 3.20 Volunteer Incidents                    | 31        |
| 3.21 Appeal Templates                       | 31        |
| <b>4. Donor Workflow</b>                    | <b>32</b> |
| 4.1 Making a Donation                       | 32        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 4.2 Understanding Donor Credit .....                              | 32        |
| 4.3 Minting a Token .....                                         | 32        |
| 4.4 Distributing a Token (Path A — Donor Controlled) .....        | 33        |
| 4.5 Letting pApAmA Distribute (Path B — PAPAMA Distributed) ..... | 33        |
| 4.6 Tracking Impact .....                                         | 33        |
| 4.7 Notifications .....                                           | 34        |
| 4.8 Reporting a Lost Token (Donor) .....                          | 34        |
| <b>5. Food Partner Workflow .....</b>                             | <b>36</b> |
| 5.1 Registering as a Food Partner .....                           | 36        |
| 5.2 Managing the Menu .....                                       | 36        |
| 5.3 Availability and Capacity Management .....                    | 36        |
| 5.4 Redeeming a Token (Serving a Meal) .....                      | 37        |
| 5.5 Uploading Proof of Service .....                              | 38        |
| 5.6 Viewing Settlements .....                                     | 39        |
| <b>6. Volunteer Workflow .....</b>                                | <b>40</b> |
| 6.1 Registering as a Volunteer .....                              | 40        |
| 6.2 Receiving and Managing Tokens .....                           | 40        |
| 6.3 Distributing Tokens to Beneficiaries .....                    | 40        |
| <b>7. For Beneficiaries — Receiving Meals .....</b>               | <b>43</b> |
| 7.1 Registering as a Beneficiary .....                            | 43        |
| 7.2 Finding a Nearby Food Partner .....                           | 43        |
| 7.3 Redeeming a Token at a Food Partner .....                     | 43        |
| 7.4 Giving Feedback .....                                         | 44        |
| <b>8. Public Features (No Account Needed) .....</b>               | <b>46</b> |
| 8.1 Public Donation — General Donation Pool .....                 | 46        |
| 8.2 UPI QR Donation .....                                         | 46        |
| 8.3 Transparency Dashboard .....                                  | 46        |
| <b>9. System Configuration Reference .....</b>                    | <b>47</b> |
| Configuration Governance .....                                    | 47        |
| Classification key .....                                          | 47        |
| 9.1 Token Settings .....                                          | 47        |
| 9.2 Meal and Redemption Settings .....                            | 47        |
| 9.3 Food Partner Settings .....                                   | 48        |
| 9.4 Settlement and Financial Settings .....                       | 49        |
| 9.5 Emergency Mode Settings .....                                 | 49        |
| 9.6 Location Settings .....                                       | 49        |
| 9.7 Quality and Security Settings .....                           | 50        |
| 9.8 Feature Toggles .....                                         | 50        |
| 9.9 Special Care Settings .....                                   | 51        |
| 9.10 Offline Capture Settings .....                               | 52        |
| 9.11 Verification, Fraud and Value Settings .....                 | 52        |

**10. Frequently Asked Questions** ..... 54

10.1 General ..... 54

10.2 For Donors ..... 58

10.3 For Food Partners ..... 60

10.4 For Administrators ..... 63

**Appendix A: Food Partner Code of Conduct** ..... 65

**Appendix B: Volunteer Code of Conduct** ..... 66

**Appendix C: Business Rules – Data Integrity and Retention** ..... 67



## How to Use This Guide

This document is the **Technical Administration Guide** for the pApAmA platform. It is written for administrators and technical stakeholders who configure, operate and oversee the platform. It covers system behaviour, administrative procedures, role definitions, configuration keys, internal route paths and integration details as implemented in the Phase 1 codebase.

A simplified **Public User Guide** — written for donors, Food Partners, volunteers and beneficiaries in non-technical language — will follow in a later phase.

Sections 1–2 cover platform overview and authentication. The **Go-Live Checklist** identifies every setting that must be reviewed before production. Sections 3–6 describe admin, donor, Food Partner and volunteer workflows respectively. Section 7 covers beneficiary interactions, Section 8 covers public features, Section 9 is the system configuration reference and Section 10 is a comprehensive FAQ.

## Terminology

| GUIDE TERM                   | APPLICATION INTERFACE TERM | MEANING                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Food Partner</b>          | Vendor                     | An approved restaurant, hotel, canteen or other food establishment participating in the platform. The application interface currently uses "Vendor" in page paths ( <code>/admin/vendors</code> , <code>/vendor/scan</code> ), configuration keys ( <code>vendor_max_complaint_rate</code> , <code>vendor_capacity_enforcement_enabled</code> ), database tables and on-screen labels. UI standardisation to "Food Partner" is a future enhancement. |
| <b>General Donation Pool</b> | Guest Pool                 | The accumulated credit balance from anonymous (guest) donations, managed by the admin and convertible into tokens for volunteer distribution.                                                                                                                                                                                                                                                                                                        |
| <b>Token</b>                 | —                          | A digital meal voucher funded by a donation, redeemable for one freshly prepared meal at a Food Partner. Tokens are not money and carry no cash value.                                                                                                                                                                                                                                                                                               |
| <b>Donor Credit</b>          | —                          | A non-withdrawable internal accounting balance representing a donor's committed meal-funding capacity. Donor Credit does not expire.                                                                                                                                                                                                                                                                                                                 |
| <b>Admin Pool</b>            | —                          | The holding area for Path B tokens awaiting volunteer allocation.                                                                                                                                                                                                                                                                                                                                                                                    |

## Conventions

- **pApAmA** — People Against Poverty and Malnutrition. This expansion is a Trust-supplied name and does not appear in the codebase.
- All page paths (e.g. `/admin/tokens`), configuration keys (e.g. `standard_token_value`) and interface labels are cited verbatim from the running application.
- Route paths, config keys and code identifiers are always shown as-is, even where they use "vendor" or "guest\_pool".

## What Changed in Version 1.2

Version 1.1 described the platform as it stood in August 2026. Between then and the end of September, the twenty-three work orders raised from this guide and the client decisions of 18 August were built. Version 1.2 records what now exists, so that a control described here as planned is genuinely still planned.

| AREA          | WHAT IS NEW IN 1.2                                                                                                                                                             |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Configuration | The ₹10 ceiling is enforced rather than advisory; every configuration change carries a reason; the dashboard names a mandatory setting that is still unset (Sections 3.13, 9). |
| Tokens        | Geographic scope from PAN-India down to a single PIN code, 60-day validity dated by distribution mode, and controlled reissue of an expired token (Section 3.3).               |
| Money         | The ₹10 contribution is tracked per redemption and gates settlement release; unspent and expired value returns to the Meal Pool rather than revenue (Sections 3.15, 3.12).     |
| Oversight     | An exception queue for inherently risky transactions, and a tamper-proof audit selection record for each cycle (Sections 3.16, 3.17).                                          |

| AREA       | WHAT IS NEW IN 1.2                                                                                                                                                                                                  |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Emergency  | An emergency is now a record with an identifier, a period and a closure reconciliation; offline capture works inside an emergency only; appeal templates need approval before dispatch (Sections 3.18, 3.19, 3.21). |
| Volunteers | Eleven one-tap incident categories, filed in two taps, with safety reports sorted first (Section 3.20).                                                                                                             |
| Privacy    | Donor notifications carry only permitted fields; volunteer screens carry the transaction minimum; every export is logged (Sections 4.7, 6.3, 3.12).                                                                 |

Seven administration screens are documented here for the first time: the contribution report, the exception queue, audit selections, the emergency register, the offline transaction register, volunteer incidents and appeal templates. Their sections are 3.15 to 3.21.

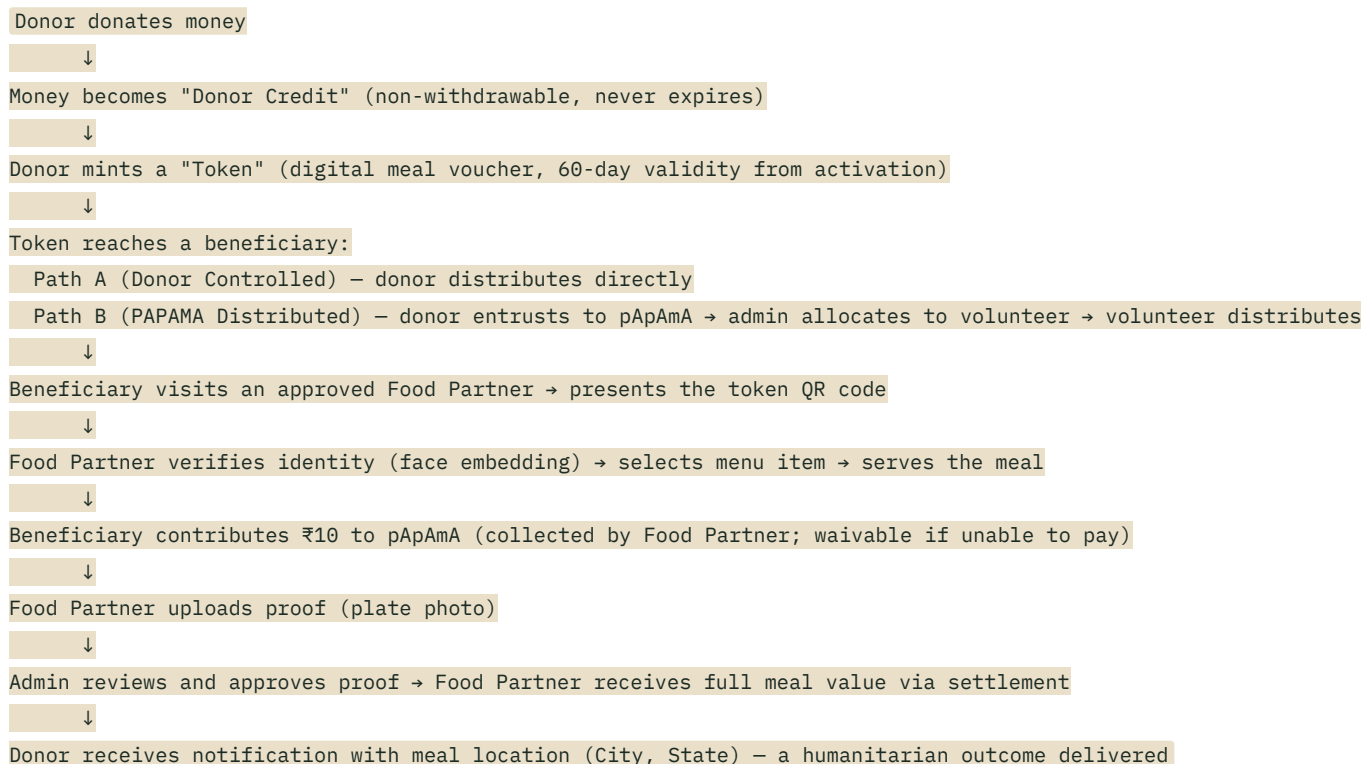
All twenty-three work orders are now built. The last one — city lock at registration (Q-5) — was held for a client decision and answered on 7 October 2026: the pilot serves Coimbatore only, and registration from another city is refused outright rather than warned.

# 1. What is pApAmA?

pApAmA (People Against Poverty and Malnutrition) is a humanitarian meal-enablement platform operated by the pApAmA Trust. Its purpose is to connect people who want to fund meals with people who need them, through a network of approved Food Partners (shown as "Vendor" in the application interface) — restaurants, hotels, canteens and other food establishments that prepare and serve fresh food.

pApAmA does not cook, store or deliver food. It enables a beneficiary to walk into an approved Food Partner and receive a freshly prepared meal, funded by a donor's contribution and verified end-to-end through digital accountability.

## How pApAmA Works



## Founding Principles

| PRINCIPLE                        | MEANING                                                                                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dignity                          | Every beneficiary is served with respect. No beneficiary is photographed or publicised. Service is discreet and equal.                                      |
| Access to Freshly Prepared Meals | Every token funds a meal that is cooked fresh and served at the Food Partner's premises. The Food Partner selects the item served from their approved menu. |
| Tokens Are Not Cash              | A token is a meal entitlement, not a financial instrument. It cannot be exchanged for money, traded, or partially redeemed for change.                      |
| Transparency and Accountability  | Every donation, token, redemption and settlement is recorded and auditable. Proof of service is required before payment.                                    |
| Humanitarian Ecosystem           | Donors, Food Partners, volunteers and administrators each play a defined role. The platform coordinates them; it does not replace them.                     |
| Mission                          | To ensure that no person in need is denied a meal when a donor has funded one and a Food Partner is ready to serve it.                                      |

## Platform Roles

| ROLE          | RESPONSIBILITY                                                                                                                                                |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Administrator | Operates the platform: approves registrations, reviews meal proofs, runs settlements, monitors fraud, configures system rules and manages emergency response. |



| ROLE                         | RESPONSIBILITY                                                                                                                                                                                                                                   |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compliance Officer</b>    | Audits platform activity with read-only access to all operational data — donations, tokens, vendors, beneficiaries, settlements, proofs, fraud flags and audit logs. Cannot create, update, approve or delete any record.                        |
| <b>Vendor Manager</b>        | Manages Food Partner relationships: approves vendor registrations and menus, manages vendor KYC documents, reviews settlements and proofs, triages complaints. Cannot approve beneficiaries or volunteers, and cannot override settlement holds. |
| <b>Donor</b>                 | Funds meals by donating, minting tokens and choosing a distribution path. Tracks the impact of donations through notifications and dashboards.                                                                                                   |
| <b>Food Partner (Vendor)</b> | Serves meals to token holders, uploads proof of service and receives payment through periodic settlements. Collects the ₹10 beneficiary contribution as an authorised collection agent of pApAmA.                                                |
| <b>Volunteer</b>             | Receives tokens from the Admin Pool and distributes them to beneficiaries in the field. Assists with beneficiary registration.                                                                                                                   |
| <b>Beneficiary</b>           | Registers for eligibility, receives tokens and redeems them for meals at approved Food Partners.                                                                                                                                                 |
| <b>Guest</b>                 | Donates without creating an account. Guest donations accumulate in the General Donation Pool (shown as "Guest Pool" in the application interface) for admin-managed distribution.                                                                |

**pApAmA does not cook, store or deliver food.** Food is freshly prepared by approved Food Partners and consumed at their premises.

## 2. How to Sign In

### Role Purposes

- **Administrator, Compliance Officer and Vendor Manager** access the admin console at `/login`. The console adapts to each role's permission scope — an administrator sees all features, a compliance officer sees read-only audit views and a vendor manager sees vendor onboarding and management.
- **Compliance Officer capabilities:** Read-all access on audit logs, fraud monitoring, donations, tokens, vendors, beneficiaries, settlements, proofs and all operational data. No mutation rights — cannot create, update, delete, approve or override anything.
- **Donors** manage their donations, tokens and impact at `/donor/login`.
- **Food Partners** (shown as "Vendor" in the application interface) manage their menus, serve meals and track settlements at `/vendor/login`.
- **Volunteers** manage their token holdings and distribution activity at `/volunteer/login`.
- **Beneficiaries** do not have a login-based dashboard. Their interaction with the platform is through the registration form and the token redemption process at Food Partners. This is a deliberate design choice for digital-literacy and device inclusion — a beneficiary needs neither a smartphone nor an account to receive a meal.
- **Guests** donate at the public `/donate` page with no account required.

### Login Pages

| ROLE                                | SIGN IN AT                    | REGISTRATION                       |
|-------------------------------------|-------------------------------|------------------------------------|
| Admin / Compliance / Vendor Manager | <code>/login</code>           | Created by system administrator    |
| Donor                               | <code>/donor/login</code>     | <code>/donor/signup</code>         |
| Food Partner (Vendor)               | <code>/vendor/login</code>    | <code>/vendor/register</code>      |
| Volunteer                           | <code>/volunteer/login</code> | <code>/volunteer/register</code>   |
| Beneficiary                         | No login                      | <code>/beneficiary/register</code> |

### Registration and Approval Flow

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| Donor:        | <code>/donor/signup</code> → Active immediately                                 |
| Food Partner: | <code>/vendor/register</code> → pending → Admin/VM review → approved → Active   |
| Volunteer:    | <code>/volunteer/register</code> → pending → Admin review → approved → Active   |
| Beneficiary:  | <code>/beneficiary/register</code> → pending → Admin review → approved → Active |
| Guest:        | <code>/donate</code> → No account needed                                        |

Each role follows a different path from registration to active use:

**Donors** — Registration is immediate. A donor signs up at `/donor/signup` with an email and password (minimum 6 characters, client-side validation only) and can begin donating straight away. No admin approval is required.

**Food Partners (Vendors)** — Registration requires admin approval. A Food Partner registers at `/vendor/register` with business details, FSSAI licence number and KYC documents. Their email is pre-confirmed server-side (no email verification step). The registration enters `pending` status and must be approved by an administrator or vendor manager before the Food Partner can sign in. KYC status is also set to `pending` at registration.

**Volunteers** — Registration requires admin approval. A volunteer registers at `/volunteer/register` with name, contact details and a face capture (only the mathematical embedding is stored; no photograph is retained). Their email is pre-confirmed server-side. The registration enters `pending` status and must be approved by an administrator.

**Beneficiaries** — Registration requires admin approval. A beneficiary (or a volunteer assisting them) submits a registration at `/beneficiary/register`. The registration enters `pending` status

and must be approved by an administrator.

## Forgot Password

Use the `/forgot-password` page to request a password reset link by email.

---

## Go-Live Checklist

Before the platform goes into production, every setting below must be reviewed and configured by an authorised administrator. This checklist distinguishes **business configuration** (set via the admin console at `/admin/system-config`) from **technical environment variables** (set by the authorised deployment team in the hosting environment).

### Business Configuration (Admin Console)

| SETTING                                                      | CLASSIFICATION            | ACTION REQUIRED                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>standard_token_value</code>                            | Mandatory                 | Set the standard meal token value (₹). Tokens cannot be minted until this is set.                                                                                                                                                                                                                                             |
| <code>meal_cooldown_hours</code>                             | Mandatory                 | Set the minimum hours between meals for a beneficiary (e.g. 6). If NULL, no cooldown is enforced.                                                                                                                                                                                                                             |
| <code>max_meals_per_day</code>                               | Mandatory                 | Set the daily meal limit per beneficiary. If NULL, no daily limit is enforced.                                                                                                                                                                                                                                                |
| <code>token_redemption_radius_km</code>                      | Mandatory                 | Set the maximum distance (km) between a Food Partner and a beneficiary for redemption.                                                                                                                                                                                                                                        |
| <code>max_tokens_per_volunteer</code>                        | Mandatory                 | Set the concurrent holding limit for volunteers. If NULL, no limit is enforced — exposure is unbounded.                                                                                                                                                                                                                       |
| <code>token_expiry_days</code>                               | Mandatory                 | Set to <b>60</b> per approved policy (60-day validity from activation — activation date varies by distribution mode: creation for Donor Controlled, distribution event for PAPAMA Distributed).                                                                                                                               |
| <code>co_contribution_max</code>                             | Mandatory                 | Set to <b>10</b> (₹10 beneficiary contribution). A hard ceiling of ₹10 is enforced in the configuration route: a higher value is refused and the attempt is recorded against the administrator who made it.                                                                                                                   |
| <code>vendor_max_complaint_rate</code>                       | Recommended               | Set the complaint-ratio threshold for auto-suspension flagging (e.g. 0.15).                                                                                                                                                                                                                                                   |
| <code>vendor_auto_suspend_enabled</code>                     | Review                    | Default OFF. The graduated corrective-action ladder governs Food Partner discipline operationally; auto-suspend remains available but OFF.                                                                                                                                                                                    |
| <code>vendor_capacity_enforcement_enabled</code>             | Recommended               | Enable if Food Partners have set daily capacity limits.                                                                                                                                                                                                                                                                       |
| <code>meal_window_enforcement_enabled</code>                 | Recommended               | Enable and configure meal windows at <code>/admin/meal-windows</code> before activating.                                                                                                                                                                                                                                      |
| <code>settlement_random_audit_rate</code>                    | Mandatory                 | Set to <b>0.10</b> (10% baseline per approved audit policy).                                                                                                                                                                                                                                                                  |
| <code>emergency_mode_enabled</code>                          | Review                    | Default OFF. Review emergency values before any activation:<br><code>emergency_max_meals_per_day</code> = 4, <code>emergency_meal_cooldown_hours</code> = 3, <code>emergency_mode_max_duration_days</code> = 7.                                                                                                               |
| <code>city_lock_enabled</code> / <code>operating_city</code> | <b>Mandatory together</b> | Set both, or neither does anything. <code>city_lock_enabled</code> on its own with no <code>operating_city</code> enforces nothing — the check cannot be evaluated and is skipped. Currently: lock <b>on</b> , city <b>Coimbatore</b> . The lock applies at registration (refused outright) and at redemption (meal refused). |
| <code>transparency_dashboard_enabled</code>                  | Review                    | Enable when public transparency page is ready.                                                                                                                                                                                                                                                                                |
| <code>proof_phash_dup_distance</code>                        | Recommended               | Set a threshold for duplicate proof-photo detection (perceptual hash distance).                                                                                                                                                                                                                                               |
| <code>audit_log_retention_days</code>                        | Review                    | Leave NULL for permanent retention (recommended).                                                                                                                                                                                                                                                                             |
| Special Care settings                                        | Review                    | Configure <code>special_care_post_delivery_months</code> if Special Care is active. See Section 9.                                                                                                                                                                                                                            |
| Notification templates                                       | Review                    | Review and customise templates at <code>/admin/notification-templates</code> .                                                                                                                                                                                                                                                |
| <code>offline_capture_enabled</code>                         | Review                    | Ships off. Leave off until the four offline limits below are set. See Section 9.10.                                                                                                                                                                                                                                           |
| <code>offline_max_pending_per_device</code>                  | <b>Client decision</b>    | Unset. How many unsynced captures one device may hold. No limit is enforced while unset.                                                                                                                                                                                                                                      |
| <code>offline_max_pending_per_volunteer</code>               | <b>Client decision</b>    | Unset. How many unsynced captures one volunteer may hold.                                                                                                                                                                                                                                                                     |
| <code>offline_max_sync_window_hours</code>                   | <b>Client decision</b>    | Unset. How long after capture a record may still sync.                                                                                                                                                                                                                                                                        |
| <code>offline_unsynced_alert_hours</code>                    | <b>Client decision</b>    | Unset. When to alert that a device has stopped syncing.                                                                                                                                                                                                                                                                       |
| <code>offline_rejected_settlement_policy</code>              | <b>Client decision</b>    | Set to <b>review</b> , which decides nothing deliberately. Choose pay, withhold, or review case by case.                                                                                                                                                                                                                      |

| SETTING                                      | CLASSIFICATION | ACTION REQUIRED                                                           |
|----------------------------------------------|----------------|---------------------------------------------------------------------------|
| <code>special_care_token_value</code>        | Review         | Set to <b>100</b> per approved policy. See Section 9.11.                  |
| <code>printed_token_area_lock_default</code> | Review         | Unset. Printed tokens take no area lock unless one is chosen at creation. |
| <code>institution_bulk_allocation_max</code> | Review         | Unset. A bulk allocation of any size is permitted.                        |

## Technical Environment Variables

| VARIABLE                         | ACTION REQUIRED                                                                                                                    |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>NEXT_PUBLIC_UPI_VPA</code> | Set the UPI merchant VPA for real payment collection. Verified by the authorised deployment team — not editable by administrators. |
| <code>TOKEN_QR_SECRET</code>     | Set the HMAC secret for token QR code generation. Must be kept confidential.                                                       |
| SMS/Email/WhatsApp API keys      | Configure when notification channels are activated (currently stub adapters).                                                      |

**Built — September 2026 (work order Q-4, B-06):** The admin dashboard now carries this readiness indicator. Where a mandatory setting is still unset, an amber banner names the key — for example `max_tokens_per_volunteer` — and the banner clears only when every mandatory value is set. **Sixteen keys are currently unset**, including the four offline limits above; the dashboard is the authoritative list at any moment.

## 3. For Admins — Running the Platform

The admin console is the operational centre of pApAmA. After signing in at `/login`, administrators, compliance officers and vendor managers land on the admin dashboard. Each user sees only the features their role permits — enforced both in the navigation (inaccessible pages are hidden) and at the API level (every route checks the permission matrix and returns HTTP 403 for unauthorised actions).

### 3.1 Admin Dashboard (Home)

Page: `/admin`

The dashboard provides an at-a-glance view of platform health through six KPI cards and supporting panels.

KPI Cards:

| CARD                   | WHAT IT MEASURES                                                | WHAT ACTION IT PROMPTS                                                       |
|------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------|
| Total Donations        | Cumulative donation value received                              | If flat or declining, review donor engagement and public donation visibility |
| Total Tokens Minted    | Count of tokens created from donor credit                       | A growing gap between donations and tokens may indicate credit sitting idle  |
| Total Redemptions      | Count of meals served through token redemptions                 | The primary mission metric — meals delivered                                 |
| Proofs Awaiting Review | Count of unreviewed meal proofs (highlighted red when non-zero) | Unreviewed proofs delay Food Partner payments; clear the queue promptly      |
| Open Fraud Flags       | Count of unresolved fraud flags (highlighted when non-zero)     | Investigate and resolve flags to maintain platform integrity                 |
| Settlements on Hold    | Count of settlements with a hold applied                        | Held settlements block Food Partner payments; review and release or escalate |

Each card links directly to its detail page.

Supporting panels:

- **Community Impact** — Aggregate numbers: total donations, meals sponsored, meals served, active Food Partners, beneficiaries reached.
- **Recent Activity** — The last 10 audited actions on the platform (who did what, when). Links to the full audit log.
- **Section Directory** — Quick links to every admin page, organised by category.

### 3.2 Managing Donations

Page: `/admin/donations`

This page lists every donation on the platform — from registered donors and from anonymous guest donations.

What the admin sees:

- All donations with amount, donor name (or "Guest"), payment method, status and date
- Client-side text search across donor label, payment reference and status (20 records per page). Export is not available on this page.
- The **General Donation Pool balance** (shown as "Guest Pool" in the application interface) — accumulated credit from anonymous donations

**General Donation Pool purpose and lifecycle:**

The General Donation Pool exists so that anyone can fund meals without creating an account. Its lifecycle has six stages:

1. **Donation** — A guest donates at `/donate` or `/donate/qz`

2. **Credit** — The donation amount is credited to the General Donation Pool balance
3. **Accumulation** — Pool credit accumulates until the admin acts
4. **Token conversion** — The admin selects an amount and mints tokens from pool credit into the Admin Pool
5. **Volunteer distribution** — The admin allocates minted tokens to volunteers for field distribution
6. **Redemption** — A beneficiary redeems the token at a Food Partner for a meal

CSR donations and institutional donations are handled through dedicated pages (`/admin/csr` and `/admin/institutions`) and follow the same underlying credit-to-token conversion mechanism.

**What the admin can do:**

- **Convert General Donation Pool credit to tokens** — select an amount and mint tokens into the Admin Pool for volunteer distribution
- **Reverse a donation** — in case of a payment error or duplicate. This reverses the internal credit; it is not a money-back refund. pApAmA does not offer money-back refunds — donated funds are committed to meals.

### 3.3 Managing Tokens

Page: `/admin/tokens`

Tokens are the core accountability instrument of pApAmA — each one traces a donor's generosity through the system to a meal served. All tokens across the platform are visible here, filterable by status.

**Token Model (approved policy):**

Per the approved token model, a pApAmA token is **PAN INDIA** by default and may be redeemed at any authorised pApAmA Food Partner anywhere in India. Only a geographic restriction specifically selected by the donor restricts the token's redemption area.

**Built — September 2026 (work order A-2, B-23):** Donor-selected geographic restriction (PAN INDIA / State / District / City / PIN) stored as a token attribute and checked at redemption against the actual service location. Token face content (type, value, geographic eligibility, activation date, expiry date) for physical and digital tokens.

Every token has a **60-day validity period** from its activation date. The activation date depends on the distribution mode (confirmed 18 Aug):

| DISTRIBUTION MODE                  | ACTIVATION DATE                                                    | RATIONALE                                                                                                                                                                                                            |
|------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Donor Controlled</b> (Path A)   | Date of token creation/issue to the donor                          | The donor holds and distributes the token; the 60-day clock starts when the token is minted.                                                                                                                         |
| <b>PAPAMA Distributed</b> (Path B) | Date the token is actually distributed/assigned to the beneficiary | The token may sit in the Admin Pool or with a volunteer before reaching a beneficiary; the 60-day clock starts only when it is distributed, not when it was created. FIFO continues to govern pool allocation order. |

A PAPAMA Distributed token that remains undistributed past its expiry period goes through the controlled reissue mechanism — a new token is created, permanently linked to the original, and the original QR becomes permanently non-redeemable.

After expiry, an unused token becomes permanently invalid and may be considered for **controlled reissue** by an authorised administrator (see below). The `token_expiry_days` configuration is set to 60.

**Built — September 2026 (work order A-2, B-23):** The per-mode activation-date rule. Validity is counted from issue for Donor Controlled tokens and from the distribution event for PAPAMA Distributed tokens, so a token waiting in the admin pool does not burn its own life.

## Distribution Mode:

Tokens are created with one of two distribution modes, fixed at creation:

| MODE                      | PATH   | DESCRIPTION                                                                | 60-DAY ACTIVATION STARTS                                     |
|---------------------------|--------|----------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>PAPAMA Distributed</b> | Path B | Token enters the Admin Pool for volunteer allocation.                      | At distribution/assignment to beneficiary (confirmed 18 Aug) |
| <b>Donor Controlled</b>   | Path A | Token goes <b>live</b> immediately for the donor to distribute personally. | At creation/issue to donor (confirmed 18 Aug)                |

**Built — September 2026 (work order A-4 and F-5, B-32):** FIFO (First-In-First-Out) allocation for pool tokens including the Special Care distribution pool. Donor-controlled tokens never enter FIFO — this is architecturally guaranteed by the distribution mode attribute.

## Token statuses:

| STATUS                       | MEANING                                                                                    | HOW A TOKEN REACHES THIS STATUS           | APPROVED VOCABULARY     |
|------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------|-------------------------|
| <b>generated</b>             | Just created, before the donor picks a distribution path                                   | Donor mints a token from credit           | Created                 |
| <b>live</b>                  | Active — donor chose to distribute personally (Path A)                                     | Donor selects "I'll distribute it myself" | Donor Controlled        |
| <b>in_admin_pool</b>         | Donor chose to let pApAmA distribute (Path B) — waiting for admin allocation               | Donor selects "Let pApAmA distribute"     | Available / FIFO Pool   |
| <b>assigned_to_volunteer</b> | Allocated to a volunteer, not yet given to a beneficiary                                   | Admin allocates from the Admin Pool       | Allocated               |
| <b>distributed</b>           | Volunteer has given the token to a beneficiary                                             | Volunteer marks as distributed            | Allocated (distributed) |
| <b>redeemed</b>              | Terminal: consumed at a Food Partner — a meal was served                                   | Food Partner completes redemption         | Redeemed                |
| <b>expired</b>               | Terminal: time ran out before the token was redeemed. Display as "Expired – Not Redeemed". | Expire sweep runs (admin-triggered)       | Expired                 |
| <b>blocked</b>               | Terminal: reported lost — replaced with a new token                                        | Admin or donor reports token as lost      | Blocked                 |

**QR payload integrity:** Each token's QR code is derived from the token ID using an HMAC-SHA256 signature with a server-held secret (**TOKEN\_QR\_SECRET**). The QR payload is deterministic and re-derivable but unguessable without the secret — a forged or altered QR code will not match any stored hash and will be rejected at redemption. Only the SHA-256 hash of the payload is persisted in the database (**tokens.qr\_hash**); the payload itself is never stored. The QR carries the token ID only — the backend is the final authority on validity, value and geographic scope.

## What the admin can do:

**Run Expire Sweep** — automatically expire all tokens past their expiry date. The expire sweep is triggered manually by an administrator rather than running automatically.

**Run Scheduled-Redemption Reminder Sweep** — send reminder notifications for tokens with upcoming scheduled redemption dates. The sweep at **/api/admin/scheduled-reminders/sweep** checks for tokens whose **scheduled\_redemption\_date** is within 7 days and sends a reminder to the relevant party. Each reminded token is marked as **reminded** to prevent duplicate notifications. This is a separate mechanism from the expire sweep — it concerns pre-arranged redemption dates, not token expiry.

## Controlled Reissue (replaces revalidation):

When a token expires without being redeemed, an authorised administrator may reissue it through a controlled process:

1. Administrator reviews the expired token and determines reissue is appropriate
2. Administrator approves the reissue with a recorded reason



3. The system creates a **new token** with a new ID, new QR code and new 60-day validity dates
4. The new token is permanently linked to the original via `replacement_for_token_id`
5. The original token remains **permanently expired** — it is never reactivated

#### The rules the system enforces on a reissue:

- The reason must be substantive. A reason shorter than ten characters is refused — "lost" is not an audit trail.
- A token can be reissued **once**. A second attempt is refused and names the replacement that already exists.
- The reissue raises a `token_reissue` exception (Section 3.16), so a second person sees it.
- The original token reads "Expired – Reissued as PPM-..." wherever it is displayed, and links to its replacement in both directions.
- The expired value moves to the Meal Pool, so the reissue is funded rather than conjured.

This controlled reissue implements the approved policy that expired token value returns to the Meal Pool for future meals rather than being written off.

**Built — September 2026 (work order F-4, B-03):** Expired and unspent value now returns to the Meal Pool with a ledger entry, and is never recorded as revenue. A ₹60 token spent on a ₹50 meal moves ₹10 to the pool; an expired token moves its whole value there. The counter screen says "returned to Meal Pool" rather than "forfeited", because that is what happens.

**Built — September 2026 (work order A-2, B-23):** The controlled reissue model as described above. The existing `token_revalidation_allowed` configuration key and associated code remain in the codebase but are **confirmed retired** (client confirmation 18 Aug) in favour of the reissue model — expired tokens are permanently non-redeemable and may only be reissued through the authorised reissue process.

#### Report a token as lost:

1. The admin (or donor — see Section 4.8) reports a token as lost
2. The old token is immediately set to `blocked` status with a `cancelled_at` timestamp
3. A new replacement token is minted with the same value, type, donor, beneficiary and expiry
4. The new token is linked to the original via `replacement_for_token_id`
5. If the replacement minting fails, the system automatically un-blocks the original token (compensating rollback) to restore the original state
6. Both the block and the replacement are audit-logged with `old_token_id` and `new_token_id`

Only tokens in `live` or `distributed` status are eligible for lost-token replacement.

#### Revoke a token:

- Only tokens in `assigned_to_volunteer` status can be revoked
- Revocation returns the token to `in_admin_pool` status for reallocation — value is preserved in the system
- No donor credit reversal and no money refund occurs
- The revocation is audit-logged with channel `admin_revoke` and an optional reason
- There is no donor-side cancellation mechanism — donors cannot cancel or revoke their own tokens

#### Token value disposition:

| SCENARIO                                                  | BEHAVIOUR SINCE SEPTEMBER 2026                                                                                                                                                                        |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Forfeited balance</b> (token value exceeds menu price) | The difference is posted to the <code>meal_pool</code> ledger, never to revenue. A ₹60 token spent on a ₹50 meal moves ₹10 to the pool. The counter screen says "returned to Meal Pool".              |
| <b>Expired tokens</b>                                     | The expire sweep posts the whole token value to the <code>meal_pool</code> ledger. The token itself stays permanently expired and can only be replaced through the controlled reissue in Section 3.3. |
| <b>Special Care surplus</b>                               | The ₹100-minus-meal difference goes to the <code>special_care_pool</code> ledger, which funds future Special Care meals only. It never mixes with the general Meal Pool.                              |
| <b>Revoked tokens</b> (volunteer-held)                    | Token status resets to <code>in_admin_pool</code> for reallocation — unchanged.                                                                                                                       |

**Built — September 2026 (work order F-4, B-03):** The approved policy above is what the platform now does. The Meal Pool balance is shown on the analytics dashboard as its own tile; there is no longer a "forfeited value" figure anywhere in the platform, because no value is forfeited.

### 3.4 Beneficiary Registration Approvals

Page: `/admin/beneficiary-registrations`

When a person registers as a beneficiary (via `/beneficiary/register` or with volunteer assistance), their application appears here for review.

**Verification objective:** The admin's role is to confirm that the applicant belongs to an eligible category and that their face capture is usable for identity verification at redemption. The approval decision is made on the basis of category, face verification and administrative judgement.

**Beneficiary categories (as implemented):**

The platform supports four beneficiary categories, hardcoded in `lib/types/enums.ts`. Adding a category requires a code change.

| CATEGORY                  | CODE VALUE                     |
|---------------------------|--------------------------------|
| Pregnant Women            | <code>pregnant_women</code>    |
| Patients                  | <code>patient</code>           |
| Persons with Disabilities | <code>disability</code>        |
| Disaster-Affected         | <code>disaster_affected</code> |

**Future categories:** The Trust intends to support additional categories in a later phase: Children, Elderly Persons, Lactating Mothers and General Category. These are not implemented today.

**What the admin sees:**

- Applicant's name, category, contact details and face capture status
- An Aadhaar column. The admin list displays a boolean presence indicator for each applicant. However, no registration path in the application collects an Aadhaar value — the underlying `aadhaar_hash` field is accepted by the API but is never populated by any form. Aadhaar is not required; face verification is the primary identity method.

**What the admin can do:**

- **Approve** — creates a verified beneficiary record; the person can now receive and redeem tokens. Eligibility expiry is calculated automatically for `pregnant_women` and `patient` categories.
- **Reject** — with a reason; the applicant can re-apply

Approved beneficiaries are listed at `/admin/beneficiaries` where the admin can view and update their profiles.

**Dignity framing:** Beneficiary registration is designed to be as simple as possible — name, contact, category, location hint and face capture. No smartphone, bank account or formal ID is required. A volunteer can assist with registration in person.

**Privacy:** Only a non-reversible 1024-dimension face embedding is retained by the platform. The face image is computed on-device and never leaves the device on which it was captured — no photograph is ever transmitted to or stored on the server.

### 3.5 Managing Food Partners

Page: `/admin/vendors`

Food Partners (shown as "Vendor" in the application interface) are mission partners — the establishments that prepare and serve meals to beneficiaries. This page manages their lifecycle from onboarding to active service.

#### Onboarding journey:

1. The Food Partner registers at `/vendor/register` with business details, contact information, geo-coordinates and their FSSAI licence number. Note that only the FSSAI licence number is captured; no expiry date is recorded.
2. The admin reviews the registration at `/admin/vendors` — checking business details, location suitability and FSSAI licence.
3. The admin reviews uploaded KYC documents at `/admin/vendors/[id]/documents`. KYC documents are held in a private storage bucket and served to the admin via short-lived signed URLs (1-hour TTL).
4. The admin approves or rejects the registration.
5. Once approved, the Food Partner can sign in, set up their menu and begin serving meals.

#### What the admin can do:

- **Register a new Food Partner** — enter details (name, location, FSSAI licence, GST, contact, geo-coordinates)
- **Approve or reject** a registration
- **Suspend** a Food Partner — temporarily block them from receiving redemptions
- **Reinstate** a suspended Food Partner
- **Manage KYC documents** — view uploaded documents at `/admin/vendors/[id]/documents`

#### Graduated Corrective-Action Ladder:

The Foundation operates a graduated discipline framework for Food Partner governance.

`vendor_auto_suspend_enabled` remains **OFF**; the ladder governs operationally:

| STEP | ACTION                               | WHEN                                                                      |
|------|--------------------------------------|---------------------------------------------------------------------------|
| 1    | <b>Warning</b>                       | First minor quality or compliance issue                                   |
| 2    | <b>Final Warning</b>                 | Repeated minor issue or first moderate issue                              |
| 3    | <b>Penalty / Enhanced Monitoring</b> | Continued issues after final warning                                      |
| 4    | <b>Suspension</b>                    | Failure to improve after enhanced monitoring                              |
| —    | <b>Immediate Suspension</b>          | Food-safety hazard, fraud, or risk to beneficiaries — bypasses the ladder |

Settlement audit finding severities (Critical / Major / Minor per Section 3.8) link to this corrective-action framework.

#### Quality monitoring:

- **Quality score composition:** Food Partner quality scores are computed from beneficiary feedback ratings, complaint rates and inspection outcomes (`lib/services/vendorRating.ts`). The complaint rate is calculated as the count of `is_complaint=true` feedback entries divided by total feedback count.
- If a Food Partner's complaint rate exceeds the threshold (`vendor_max_complaint_rate` in System Config) and auto-suspend is enabled (`vendor_auto_suspend_enabled`), they are automatically flagged. The complaint ratio should be read in context — a single complaint from very few interactions may not indicate a systemic issue.
- **Vendor inspection workflow:** Administrators can record surprise inspections at `/admin/vendor-feedback` via `POST /api/admin/vendor-inspections`. Each inspection records pass/fail and notes. On a failed inspection, the `vendor_inspection_fail_penalty` configuration value is read as a **numeric quality-score deduction** applied to the Food Partner's quality score:  $\text{nextScore} = \max(0, \text{round}((\text{current\_score} - \text{penalty}) * 100) / 100)$ . The penalty is audit-logged. If `vendor_inspection_fail_penalty` is not set, no penalty is applied (soft-skip).

### 3.6 Menu Approvals

Page: `/admin/vendor-menus`

Food Partners (shown as "Vendor" in the application interface) propose menu items with pricing. Each item must be approved by an administrator or vendor manager before it can be used for redemptions. Menu approval ensures that the items and prices offered to beneficiaries meet platform standards.

**Menu dignity:** Beneficiaries are served from the same approved menu as paying customers. The platform does not contemplate a separate or lower grade of food for token-funded meals.

**What the admin can do:**

- View proposed menu items (name, description, price, Food Partner)
- **Approve** — the item becomes available for meal redemptions
- **Reject** — with a reason; the Food Partner can modify and resubmit. The purpose of rejection is to reach an approved menu, not to penalise. A good rejection reason is specific and actionable — it tells the Food Partner exactly what to change (e.g. "price exceeds the standard token value" or "item description too vague — please specify portion size") so they can correct and resubmit promptly.

**Special Care items:** Food Partners may propose local equivalents for Special Care categories (pregnant women, patients). The categories with Special Care equivalents are defined by the `is_special_care_equivalent` flag on menu items. Approve these on a case-by-case basis.

Note that menu price edits overwrite the previous value with no price history retained, and there is no per-item availability toggle.

---

### 3.7 Reviewing Meal Proofs

Page: `/admin/proofs`

After a Food Partner serves a meal, they upload a plate photo as proof of service. The admin must review and approve the proof before the Food Partner's payment is released.

**Filter tabs:** Awaiting Review | Approved | Rejected | All

**For each proof submission, the admin sees:**

- Food Partner name and beneficiary category
- Menu item served and its value
- Plate photo

**Proof review checklist:**

- The plate photo should show the meal served, not the beneficiary (dignity and privacy)
- The photo should be clear, well-lit and show recognisable food
- The menu item claimed should be consistent with what appears in the photo

**What the admin can do:**

- **Approve** — releases the payment for settlement processing. A notification is sent to the donor who funded the token.
- **Reject** — a reason is mandatory (this is the only action in the platform that enforces a mandatory reason server-side). The Food Partner sees the rejection and can re-upload. Payment stays locked until approved. No resubmission deadline exists.

**Duplicate-media detection (phash flow):**

When `proof_phash_dup_distance` is configured, the system computes a perceptual hash (phash) of each uploaded proof photo and compares it against existing proofs. If a match is found:

1. The proof remains in `submitted` status awaiting admin review — it is **not auto-rejected**
2. A `duplicate_media` fraud flag is raised (with `blocked: false`)
3. Settlements covering the related redemptions are automatically placed on hold

4. Detection is best-effort (soft-fail) — a detection failure does not block proof submission

**Important:** A duplicate-media flag is a signal for investigation, not proof of fraud. The same plate, crockery or setting may legitimately appear in multiple photos. The admin investigates and determines the appropriate response.

**Fraud patterns to watch for:** The platform's fraud module detects several patterns automatically, but proof reviewers should also watch for:

- **Identical backgrounds** — multiple proof photos with the same table, crockery or setting that suggest staging rather than distinct meals
- **Inconsistency between claimed item and photo** — a plate photo that does not match the menu item selected at redemption
- **Unusual volume** — a high number of proofs from a single Food Partner in a short period

**Important:** Proof photos are write-once — once uploaded, they cannot be altered or deleted by the Food Partner. This ensures the integrity of the proof record.

### 3.8 Settlements and Food Partner Payouts

Pages: `/admin/settlements` and `/admin/settlement-audit`

Settlements exist to convert approved meal service into Food Partner payments in a controlled, auditable process. The Foundation's operating principle is that no individual should be able to prepare, independently verify, approve and release the same settlement without appropriate oversight.

**Payout formula:** For each redeemed meal, the platform pays the Food Partner the **full approved meal value**: `min(token_value, menu_value)`. If the menu price exceeds the token value, the difference (`menu_value - token_value`) is borne by the beneficiary as a top-up.

The ₹10 beneficiary contribution is an entirely separate transaction — it is collected by the Food Partner at the counter and remitted to the pApAmA Administration Account. It is **not** deducted from the settlement payout. The Food Partner receives the full meal value via settlement regardless of whether the ₹10 contribution was collected or waived.

**Built — September 2026 (work order F-1, B-01):** Settlement-release gate on ₹10 contribution reconciliation — settlements will only be eligible for release after the contribution has been received/reconciled or an authorised humanitarian waiver has been recorded against each redemption.

#### Settlement lifecycle:

The verified settlement lifecycle follows this progression with an orthogonal hold/release mechanism:

pending → locked → approved → reconciled → paid  
 ↑  
 hold/release at any point before paid

| ACTION    | STATUS TRANSITION                                                | AUDIT-LOGGED                                 | NOTES                                                     |
|-----------|------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------|
| Generate  | Creates settlement header + line items                           | Yes                                          | Groups approved-but-unsettled redemptions by Food Partner |
| Lock      | pending → locked                                                 | Yes ( <code>settlement.lock</code> )         | Settlement is prepared for review                         |
| Unlock    | locked → pending                                                 | Yes ( <code>settlement.unlock</code> )       | Returns to pending for amendment                          |
| Approve   | locked → approved                                                | Yes ( <code>settlement.approve</code> )      | Approved for reconciliation                               |
| Reconcile | approved → reconciled                                            | Yes<br>( <code>settlement.reconcile</code> ) | Pre-payment reconciliation complete                       |
| Hold      | <code>on_hold = true</code> with optional <code>hold_note</code> | Yes                                          | Suspends processing at any point before paid              |
| Release   | <code>on_hold = false</code>                                     | Yes                                          | Resumes processing                                        |

| ACTION | STATUS TRANSITION                           | AUDIT-LOGGED         | NOTES                                     |
|--------|---------------------------------------------|----------------------|-------------------------------------------|
| Pay    | reconciled → paid with settled_at timestamp | Yes (settlement.pay) | Ledger entry posted; settlement finalised |

### Maker-checker discipline:

The Foundation operates a maker-checker principle for settlements: the person who prepares a settlement should not be the same person who independently approves and releases it. This is currently an **operating procedure** enforced through administrative practice and audit oversight.

**Built — September 2026 (work order F-2, B-24):** System-enforced maker-checker segregation — including blocked same-user prepare/approve, settlement versioning with approval auto-invalidation on material change, reject/return-to-maker with mandatory reason, bank-account change four-eyes control, and three-way reconciliation view including ₹10 contribution figures. A live demonstration of the maker-checker workflow (10-step happy path + 5 exception scenarios) is required before final sign-off.

### Hold discipline:

When a settlement is placed on hold, the `hold_note` field records the reason (optional in the current system). The responsible person is recorded via `actor_id` in the audit log. Holds block Food Partner payments — review and release or escalate promptly.

**Settlement finality:** Once a settlement is marked as `paid`, the record is final. Corrections to paid settlements are recorded as separate adjustment/recovery records.

**Planned (B-07):** Auditable settlement adjustment and recovery records — corrections as new linked transactions; paid settlements never edited.

**Planned (B-08):** Line-item settlement hold — hold only the disputed transaction while protecting the Food Partner's cash flow on undisputed items.

### Settlement audit queue (/admin/settlement-audit):

A random sample of settlements is pulled for audit review. The `settlement_random_audit_rate` is set to 10% — this is the baseline, not the ceiling.

The approved audit policy: "An initial random audit rate of 10% of eligible Food Partner settlement batches shall apply during the Coimbatore pilot, subject to a minimum of one settlement per audit cycle. The system shall select the sample randomly and maintain an auditable record of the selection. Random sampling shall be supplemented by risk-based and exception-based audits. Settlements or Food Partners exhibiting defined risk indicators, material discrepancies, prior audit findings, unusual transaction patterns or compliance concerns may be subjected to enhanced or 100% review."

The audit queue includes clear/flag actions, and flagged settlements are automatically held. Selection records are maintained.

**Built — September 2026 (work order F-3, B-25):** Full risk-based audit framework — tamper-proof system selection with permanent 14-field audit records, three tiers (normal 10% / high-risk enhanced / critical 100%), exception auto-flag queue for inherently risky transactions, and finding severities linked to the corrective-action ladder.

**Note:** There is no Food Partner-side dispute or query mechanism for settlements. Food Partners have read-only access to their settlement status.

### Operational Service Levels:

The Foundation operates the following service levels as configurable parameters (not system-enforced):

| SERVICE                   | TARGET                          |
|---------------------------|---------------------------------|
| Proof approval            | ≤ 24 hours                      |
| Settlement reconciliation | ≤ 48 hours after proof approval |

| SERVICE                   | TARGET                                |
|---------------------------|---------------------------------------|
| Food Partner payment      | ≤ 7 working days after reconciliation |
| Complaint acknowledgement | ≤ 24 hours                            |
| Complaint resolution      | Normally ≤ 7 working days             |

These are Foundation operating parameters — the system does not currently enforce or alert on SLA breaches.

### 3.9 Volunteers and Token Allocation

Page: `/admin/volunteers`

Volunteers are custodians of donor trust. They receive tokens from the Admin Pool and distribute them to beneficiaries in the field — the human link between a donor's generosity and a beneficiary's meal.

**Token accountability lifecycle:**

1. The admin allocates tokens from the Admin Pool to a volunteer
2. The volunteer holds the tokens (tracked as `assigned_to_volunteer`)
3. The volunteer distributes each token to a beneficiary (status changes to `distributed`)
4. Each distribution is logged with a timestamp and channel

**Holding limits:** The system enforces a concurrent holding limit (`max_tokens_per_volunteer` in System Config). A volunteer cannot hold more than this many undistributed tokens at once. This limit exists to bound exposure — if a volunteer becomes unreachable, the maximum number of tokens at risk is capped.

**Important:** A volunteer cannot return an undistributed token to the Admin Pool. Only an admin can revoke a token back to the pool.

**Judgement factors for allocation:** When deciding how many tokens to allocate to a volunteer, the admin should weigh:

- **Distribution history** — visible at `/admin/volunteer-activity`, showing past distribution volume and patterns
- **Current holding** — the volunteer's current undistributed count against the configured `max_tokens_per_volunteer` limit
- **Area coverage** — whether the volunteer serves an area with unmet need. Note that zone-based allocation is not enforced (`volunteer_zones_enabled` is a configuration seam only; zone assignment is available but not enforced at distribution).

**What the admin can do:**

- **View all volunteers** — see their status, zone assignment and current token holding count
- **Allocate tokens** — assign tokens from the Admin Pool to a volunteer, subject to the holding limit
- **Review volunteer requests** — volunteers can request tokens via their dashboard. The admin can grant (fully or partially) or deny requests.
- **View volunteer activity** — at `/admin/volunteer-activity`, see distribution logs (when, where, how many tokens distributed)
- **Approve or reject** volunteer registrations

### 3.10 Fraud Monitoring

Page: `/admin/fraud`

The fraud monitoring system exists to protect the platform's integrity — its objective is to protect, not to suspect. Automated detection flags patterns that may indicate misuse; the admin investigates each flag and determines the appropriate response.

### Types of fraud flags:

- **Duplicate token scan** — same token QR scanned more than once
- **Face hash repeat** — same face detected at multiple Food Partners within the cooldown window
- **Cooldown breach** — beneficiary attempted to redeem before their cooldown period expired
- **Duplicate media** — same proof photo (by perceptual hash) uploaded for different redemptions. Flagged for review with `blocked: false`; related settlements auto-held. Never auto-rejected — the admin investigates and decides.

Severity levels: Low | Medium | High | Critical

### Investigation workflow:

1. Review the flag details — what triggered it, when and at which Food Partner
2. **Investigate** — mark the flag as "investigating" while gathering facts
3. Determine the outcome: - **Resolve** — confirmed issue; record the resolution with notes - **Dismiss** — false positive; record why it was dismissed

**Fair handling of false positives:** The system may flag legitimate activity — for example, a beneficiary visiting two nearby Food Partners within the cooldown window due to the first being unable to serve. Investigate before acting. Dismissal with notes is the correct handling for a false positive.

The admin can also **run a fraud scan** manually to trigger a check for duplicate patterns across recent activity.

## 3.11 Emergency Mode

Page: `/admin/emergency`

Emergency Mode is a humanitarian response mechanism — a temporary, authorised operating mode activated by pApAmA Administration in response to an approved humanitarian emergency. Its purpose is to ensure that people affected by disasters, crises or emergencies receive food assistance without being obstructed by normal-period controls that are inappropriate during the emergency.

### The three-mode distinction:

| MODE                        | DESCRIPTION                                                                                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Normal Operations</b>    | All standard controls active. Default state.                                                                                                                                  |
| <b>Emergency Mode</b>       | Meal-frequency and cooldown parameters relaxed within approved limits for the defined emergency scope. Time-boxed.                                                            |
| <b>Financial Governance</b> | Always on — in both Normal and Emergency modes. Token expiry, settlement controls, audit logging, Food Partner accountability and all financial controls remain fully active. |

### Emergency Mode values (approved):

| PARAMETER                                     | EMERGENCY VALUE      | NORMAL COMPARISON                            |
|-----------------------------------------------|----------------------|----------------------------------------------|
| <code>emergency_max_meals_per_day</code>      | 4 meals/day          | <code>max_meals_per_day</code> (admin-set)   |
| <code>emergency_meal_cooldown_hours</code>    | 3 hours              | <code>meal_cooldown_hours</code> (admin-set) |
| <code>emergency_mode_max_duration_days</code> | 7 days (auto-revert) | N/A                                          |

These values are configurable within approved limits. Extension requires explicit authorised action with a recorded reason and revised end date. No indefinite Emergency Mode is permitted.

**Flexibility, not unlimited:** Emergency Mode is a set of controlled relaxations, not an absence of controls. The principle is that humanitarian need during a genuine emergency justifies increased meal access within defined, auditable boundaries.

### How to activate:

1. Go to `/admin/emergency`



## 2. Toggle Emergency Mode ON (`emergency_mode_enabled`)

3. The system applies emergency overrides to meal limits and cooldowns

### What Emergency Mode changes:

- Meal cooldown periods become soft warnings rather than hard blocks (using `emergency_meal_cooldown_hours`)
- Daily meal limits become soft warnings rather than hard blocks (using `emergency_max_meals_per_day`)
- Emergency tokens can be issued directly into the Admin Pool for immediate volunteer distribution

### What Emergency Mode does NOT override (these remain mandatory):

- Token expiry and validity
- Donor geographic restrictions
- Token value
- Food Partner suspension status
- Food safety, FSSAI and compliance holds
- Fraud blocks
- Meal window enforcement (if enabled)
- Food Partner opening state (`is_open`)
- Food Partner stock exhaustion (`stock_exhausted`)
- Food Partner temporary closure (`temporary_closure_until`)
- Food Partner daily capacity limits (if enforced)
- All financial controls (settlements, audit, ledger entries)

### Verification relaxation during emergencies:

During an authorised Emergency Mode, pApAmA may relax beneficiary documentation and verification requirements to ensure that genuine beneficiaries are not denied emergency food assistance due to lack of documentation. The core transaction controls (valid token, not already redeemed, within validity, authorised active Food Partner, geographic restriction, emergency meal limit, cooldown, date/time recorded) continue to apply.

A face embedding may be captured where operationally feasible and appropriately consented, for transaction verification and post-emergency audit, but inability or refusal to provide a face embedding does not by itself prevent emergency meal access.

**Built — September 2026 (work order E-2 and E-3, B-27):** Face-verification skip path during active emergency, with verification level recording (Standard / Enhanced / Referred). Automatic ₹10 waiver rule tied to emergency state and scope.

**Documentation correction:** The platform stores no photograph of any person in any mode. Identity verification uses only an on-device-computed mathematical representation (1024-dimension face embedding). No image is transmitted or retained. No facial-recognition expansion is introduced by the emergency process.

### ₹10 contribution waiver during emergencies:

During an authorised Emergency Mode, pApAmA Administration may waive the ₹10 beneficiary contribution, either generally within the defined emergency scope or for specified emergency circumstances. Where the contribution is waived, the Food Partner continues to receive the full approved meal value through the normal settlement process. Every waiver is recorded against the relevant redemption transaction and included in settlement and emergency reconciliation.

**Built — September 2026 (work order E-2 and E-3, B-27):** System-indicated emergency waiver rule — waiver is never Food Partner or volunteer discretion. All transactions processed under relaxed verification and all ₹10 waivers are tagged to the relevant Emergency ID.

## Emergency ID and governance:

**Built — September 2026 (work order E-1 and E-6, B-26):** Full Emergency Response Framework — unique Emergency ID (e.g., TN-FLOOD-2026-001) with authorised activation, reason, geographic scope, beneficiary scope, period and complete audit trail. Emergency Appeal workflow with approved templates and authorised dispatch. Emergency-ID tagging of donations and tokens. Closure reconciliation (funds received/utilised/committed; tokens issued/redeemed/unused; surplus; utilisation decision; approver; closure date) as a permanent record. **Channel-extensibility requirement (confirmed 18 Aug):** The Phase 1 appeal architecture must be designed so that Phase 2 channels (SMS via DLT registration, WhatsApp via Business API) can be integrated later without fundamental redesign of the dispatch, template or tracking infrastructure.

### Surplus hierarchy (approved policy):

Any surplus funds or unused token value remaining after emergency requirements have been met:

1. First considered for the specific emergency's continuing needs
2. Then for other genuine humanitarian needs in the same affected area
3. Then transferred to the PAPAMA Emergency Response Fund or another approved humanitarian purpose

No donor refund mechanism exists for emergency contributions. This policy is disclosed at contribution time.

### What is audit-logged today:

| EVENT                                                       | WHAT IS CAPTURED                                           | DETAIL LEVEL                                                                                                  |
|-------------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Emergency toggle<br>( <code>emergency_mode_enabled</code> ) | Actor, timestamp, previous/new value                       | Via generic config path — no dedicated reason field                                                           |
| Emergency override activation                               | Actor, config key, from/to values, optional reason, expiry | Per-override detail                                                                                           |
| Emergency override manual revert                            | Actor, config key                                          | Per-override detail                                                                                           |
| Emergency auto-revert (pg_cron)                             | Aggregate count, source "pg_cron"                          | <b>Aggregate only</b> — <code>actor_id: null</code> , summary: "auto-reverted N emergency override(s) [cron]" |
| Emergency token issuance                                    | Actor, optional reason                                     | Per-token detail                                                                                              |

**Planned (B-19):** Per-override detail in auto-revert audit log (currently aggregate count only).

**Planned (B-09):** Emergency Mode governance workflow — initiator/approver/oversight roles, mandatory reason capture on activation/deactivation, extension review.

**Post-emergency review:** After Emergency Mode ends (by expiry or manual deactivation), a review of all emergency-period transactions should be conducted. Transactions processed under relaxed verification are available for risk-based post-emergency audit.

**Auto-revert:** When `emergency_mode_max_duration_days` is set, Emergency Mode automatically turns off after that many days via a pg\_cron job that runs daily at 03:00 UTC. The auto-revert is logged as an aggregate audit entry.

## 3.12 Analytics and Reports

**Analytics:** `/admin/analytics`

Visual dashboards answering key operational questions:

| DASHBOARD              | BUSINESS QUESTION IT ANSWERS                                                          |
|------------------------|---------------------------------------------------------------------------------------|
| Meals served over time | Is the platform's mission output growing? Are there seasonal or day-of-week patterns? |
| Donation trends        | Where is funding coming from? Which payment methods are used?                         |

| DASHBOARD                | BUSINESS QUESTION IT ANSWERS                                                                                                                                                |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Food Partner performance | Which Food Partners are most active? Which have quality concerns?                                                                                                           |
| Token utilisation        | What fraction of minted tokens are redeemed vs expired? Is there waste?                                                                                                     |
| Financial summary        | How much has been donated and settled, and how much unspent value sits in the Meal Pool? Unspent and expired value is shown as returned to the Meal Pool, never as revenue. |
| Fraud summary            | What types of flags are most common? Are they concentrated at specific Food Partners?                                                                                       |
| Beneficiary breakdown    | How are beneficiaries distributed across categories?                                                                                                                        |

#### Reports: </admin/reports>

Generate and export compliance and CSR reports. Reports can be downloaded as files for record-keeping.

#### Audit Logs: </admin/audit-logs>

Every admin action is logged permanently — who did what, when and on which record. This log is append-only at database level — two PostgreSQL triggers (`audit_logs_no_update`, `audit_logs_no_delete`) block any update or delete operation, including by `service_role`. Entries cannot be edited or deleted by any route, service or administrative action.

Client-side text search is available across action, entity table and summary fields (20 records per page). Export is not available on audit logs.

**Permanent-retention policy:** The `audit_log_retention_days` configuration key exists but is intentionally unset (NULL). No automated purge or retention job runs. Financial, token, settlement and governance records are never deleted. This is the approved retention policy.

### 3.13 System Configuration

#### Page: </admin/system-config>

This is where authorised administrators control the rules that govern the platform. Settings are organized in tabs.

See Section 9 for a complete reference of all configuration keys, including classification, NULL semantics and business implications.

#### Governance principles:

- Only authorised administrators may change system configuration values
- Every change is audit-logged with the previous value, new value, actor and timestamp
- **Built — September 2026 (work order Q-3, B-14):** Optional reason field on system-config change audit trail — enabling administrators to record why a significant configuration change was made
- Significant changes (e.g. token value, contribution policy, emergency activation) should follow the Foundation's internal approval practice

#### Key settings you'll use most often:

| SETTING                               | WHAT IT CONTROLS                                | GO-LIVE ACTION                          |
|---------------------------------------|-------------------------------------------------|-----------------------------------------|
| <code>standard_token_value</code>     | The value of a standard meal token (in ₹)       | Must be set before tokens can be minted |
| <code>token_expiry_days</code>        | Days before an unused token expires             | Set to 60 (approved policy)             |
| <code>max_tokens_per_volunteer</code> | Maximum tokens a volunteer can hold at once     | Must be set to bound exposure           |
| <code>meal_cooldown_hours</code>      | Minimum hours between meals for a beneficiary   | Must be set                             |
| <code>max_meals_per_day</code>        | Maximum meals a beneficiary can receive per day | Must be set                             |
| <code>emergency_mode_enabled</code>   | Toggle emergency/disaster mode ON or OFF        | Review emergency values first           |

| SETTING                                   | WHAT IT CONTROLS                              | GO-LIVE ACTION             |
|-------------------------------------------|-----------------------------------------------|----------------------------|
| <code>co_contribution_max</code>          | Beneficiary contribution amount at redemption | Set to 10 (₹10 policy)     |
| <code>settlement_random_audit_rate</code> | Fraction of settlements sampled for audit     | Set to 0.10 (10% baseline) |

## 3.14 Other Admin Sections

### Operations Management

| PAGE                                | WHAT IT DOES                                                                                                                                                                                                                                                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/admin/meal-windows</code>    | Set serving time windows (breakfast, lunch, dinner, snack). If <code>meal_window_enforcement_enabled</code> is ON, redemptions are blocked outside these windows.                                                                                                  |
| <code>/admin/vendor-capacity</code> | Monitor and set each Food Partner's daily meal capacity. Shows real-time capacity usage (served/remaining) for all Food Partners. If <code>vendor_capacity_enforcement_enabled</code> is ON, redemptions stop when a Food Partner reaches their limit for the day. |
| <code>/admin/emergency</code>       | Activate or deactivate Emergency Mode (see Section 3.11).                                                                                                                                                                                                          |
| <code>/admin/vendor-feedback</code> | View beneficiary feedback, Food Partner ratings and inspection results. Record surprise inspections.                                                                                                                                                               |

### Partnership Management

| PAGE                             | WHAT IT DOES                                                                          |
|----------------------------------|---------------------------------------------------------------------------------------|
| <code>/admin/institutions</code> | Manage partner institutions — allocate tokens in bulk for institutional distribution. |
| <code>/admin/csr</code>          | Manage corporate CSR donors and view CSR-specific reports.                            |
| <code>/admin/ngo-partners</code> | Reference registry of partner NGOs and organisations.                                 |

### Communication Management

| PAGE                                       | WHAT IT DOES                                                                                                                                                            |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/admin/notification-templates</code> | Edit the notification messages sent to donors (e.g. when their token is redeemed). Uses placeholders like <code>{{token_value}}</code> , <code>{{vendor_name}}</code> . |
| <code>/admin/complaints</code>             | View and resolve complaints submitted by beneficiaries. Status flow: Open → Investigating → Resolved/Dismissed.                                                         |

### Governance and Service Quality

| PAGE                                 | WHAT IT DOES                                                    |
|--------------------------------------|-----------------------------------------------------------------|
| <code>/admin/vendor-feedback</code>  | View beneficiary feedback and Food Partner inspection results.  |
| <code>/admin/audit-logs</code>       | Full audit trail of all admin actions (append-only, immutable). |
| <code>/admin/settlement-audit</code> | Random-sample audit queue for settlement review.                |

## 3.15 Contribution Report

Page: `/admin/contribution-report`

Every redemption carries a ₹10 beneficiary contribution with a status of **collected**, **waived** or **outstanding**. This report is the position across all of them, with the outstanding total shown first — that figure is money the platform is owed, and it is the number to watch.

The contribution belongs to pApAmA. It is never deducted from the Food Partner's payout: the partner is settled the full meal value whether the contribution was collected, waived or is still outstanding.

**Why a settlement can be blocked here:** a settlement cannot be released while any of its lines has an unresolved contribution. The blocking lines are named on the settlement itself, so an administrator can see exactly which transactions are holding the payment.

**Waivers** are recorded with who authorised them and why. A waiver applied automatically by an active emergency is recorded as such — see Section 3.18. A waiver outside the usual pattern raises an `unusual_waiver` exception (Section 3.16).

`contribution_remittance_cycle_days` sets how often collected contributions are remitted. It is currently 1.

### 3.16 Exception Queue

Page: `/admin/exception-queue`

Some transactions are risky by their nature rather than because anything has gone wrong. Rather than relying on an administrator to remember to look at them, they raise themselves into this queue.

What lands here:

| EXCEPTION TYPE                                                                                           | RAISED WHEN                                                |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <code>token_reissue</code>                                                                               | An expired token is reissued (Section 3.3)                 |
| <code>bank_account_change</code>                                                                         | A Food Partner's bank account is changed                   |
| <code>unusual_waiver</code>                                                                              | A contribution waiver falls outside the normal pattern     |
| <code>offline_duplicate</code>                                                                           | The same token is captured offline on more than one device |
| <code>manual_adjustment</code> , <code>manual_entry</code> , <code>reversal</code> , <code>refund</code> | A figure is changed by hand rather than by the system      |
| <code>fraud_linked</code>                                                                                | The transaction is connected to an open fraud flag         |
| <code>emergency_pattern</code>                                                                           | Activity during an emergency matching a flagged pattern    |

Status flow: Open → In review → Cleared or Escalated.

**Clearing requires a note.** An exception cannot be dismissed silently; the administrator records what they found. The same event cannot queue twice — a retried write does not inflate the queue or the statistics drawn from it.

**After an emergency,** this queue is filtered by Emergency ID to produce the post-emergency review described in Section 3.18.

### 3.17 Audit Selections

Page: `/admin/audit-selections`

Each audit cycle records its own selection, and that record cannot be quietly undone. Once a settlement is drawn into a sample, its row cannot be removed.

**What each selection record holds:** the cycle reference, the method used, the population it drew from, the sample it produced, the rate applied, and who drew it and when.

Methods:

| METHOD                | USE                                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------|
| <code>random</code>   | The baseline sample, at <code>settlement_random_audit_rate</code> (currently 0.10 — 10%)  |
| <code>enhanced</code> | A higher rate for a cycle that warrants closer attention                                  |
| <code>targeted</code> | A specific selection, which should carry a written reason — a targeted sample needs a why |

**The minimum of one:** where a cycle has any eligible settlements at all, at least one is always selected. A cycle can never audit nothing because a percentage rounded down.

### 3.18 Emergency Register

Page: `/admin/emergencies`

An emergency is a **record**, not a switch. (The older `/admin/emergency` page described in Section 3.11 toggles the operating mode; this register is what makes each emergency accountable.)

**What an emergency record carries:** an identifier in the form `TN-FL00D-2026-001`, a title and reason, its period — it begins when it is declared and ends at the stated end date — its

geographic scope, the estimates it was declared against, and the two relaxations it enables.

**The two relaxations, both recorded on the record itself:**

| FLAG                                         | EFFECT                                                                                                                        |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>contribution_waiver_enabled</code>     | The ₹10 is waived automatically by the system for transactions in scope — never at a Food Partner's or volunteer's discretion |
| <code>verification_relaxation_enabled</code> | The face step may be skipped, with the verification level recorded against the transaction                                    |

Every donation, token and redemption during an emergency is traceable to its Emergency ID, and an emergency token cannot be issued without an active record.

**Extending** an emergency requires a written reason and a new end date. Both are mandatory — an extension without a reason is refused.

**Closing** one requires a decision about any surplus raised for it:

| SURPLUS DECISION                       | MEANING                                         |
|----------------------------------------|-------------------------------------------------|
| <code>same_emergency</code>            | Held for continued work on the same emergency   |
| <code>continuing_need_same_area</code> | Applied to ongoing need in the same area        |
| <code>emergency_response_fund</code>   | Returned to the general emergency response fund |

Closure writes a reconciliation record and populates the post-emergency review queue with flagged transactions only — rapid repeats, reused identifiers, volume and waiver anomalies. Ordinary transactions from the emergency stay under the normal 10% random audit rather than being re-examined wholesale.

**Offline captures are reviewed separately by source.** A Food Partner recording at their own till and a volunteer recording in a field are different risk profiles, so each is measured against its own peers rather than pooled. A volunteer device holding three of five volunteer captures is 60% of its cohort and is flagged; pooled into an emergency of forty captures it would be 7% and would disappear. The closure response reports a count for each source, never a single total, and every flag in the queue is prefixed `[food_partner]` or `[volunteer]`.

**What the offline review looks for:** one device carrying most of its cohort, a device whose captures are mostly refused, a device waiving almost every contribution, the same beneficiary identifier on more than one capture, and a capture that synced long after it was taken.

### 3.19 Offline Transaction Register

Page: `/admin/offline-transactions`

When the network drops during an emergency, a Food Partner's counter can record the meal on the device and sync it later. This register is where those captures arrive.

**A capture is never a completed redemption.** It syncs as **Pending Offline Validation** and becomes a redemption only once the server re-checks it — expiry, geographic scope, token status and Food Partner standing — and an administrator approves it.

**Outcomes on sync:**

| OUTCOME                                 | MEANING                                                                                                                                                                                                                                                                           |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pending_offline_validation</code> | Accepted for validation                                                                                                                                                                                                                                                           |
| <code>duplicate</code>                  | The same token was captured more than once. <b>Both sides are flagged and neither is accepted</b> — the system cannot tell a crash retry from two volunteers helping one person from deliberate reuse, so a human decides. An <code>offline_duplicate</code> exception is raised. |
| <code>rejected</code>                   | Refused, with the reason recorded against the capture                                                                                                                                                                                                                             |

**Reasons a capture is refused:** it falls outside the emergency period; the device clock is in the future; the scanned QR matches no token; or the device has been reported compromised.

**What the device stores:** a one-way fingerprint of the token, never the redeemable code. A lost phone yields nothing usable.

**Both routes are open.** The Food Partner till is the primary route; a volunteer records in the field at `/volunteer/offline` when there is no connected Food Partner. The two share one panel and one set of server controls, and the source is stamped from the session at sync, so a device cannot claim to be the other kind. A volunteer holds a narrow `offline_capture` capability and **still cannot redeem a token online** — the two permissions are deliberately separate.

**It ships switched off.** `offline_capture_enabled` is `false` by default, and the panel does not appear on either screen until it is switched on and an emergency is active. An exception that is on by default is not an exception.

`offline_rejected_settlement_policy` decides what happens to a rejected capture at settlement. It is currently `review`, which deliberately decides nothing — the real choice between paying, withholding and reviewing case by case is the client's.

---

### 3.20 Volunteer Incidents

Page: `/admin/volunteer-incidents`

A volunteer standing in front of someone hungry needs somewhere to put a problem in two taps. This queue is where those reports arrive.

**The eleven categories**, in the client's order: no phone · no token · partner closed · partner refusing a valid token · no food · connectivity failure · token problem · urgent need · food safety concern · safety concern · other.

**The category is the only required field.** The note is optional by design. Requiring a note would push a volunteer into typing while a queue waits, and the report that never gets filed is usually the one that mattered most.

**Safety reports sort to the top** of this queue regardless of age, and stay there until they are dealt with.

---

### 3.21 Appeal Templates

Page: `/admin/appeal-templates`

An appeal goes out to donors during an emergency. Because it goes out in pApAmA's name, it cannot be sent by accident.

**The sequence:** a template is created as a **draft**, an authorised administrator **approves** it, and only then can it be dispatched. Marking a template as the pre-approved **instant** template is refused before approval and permitted after it.

**The instant template** exists so that a genuine emergency is not delayed by an approval cycle. Using it raises a post-send review task, so speed does not cost oversight.

**Audience** is one of `all`, `individual` or `csr`. A template can also be **retired** when it is no longer appropriate.

**Donations that arrive from an appeal carry the Emergency ID**, so the money raised by an appeal is traceable to the emergency it was raised for.

**Channels:** dispatch is built channel-abstract. Adding SMS or WhatsApp in Phase 2 is a new adapter rather than a redesign — see the note on DLT and WhatsApp Business registration in Section 9.8.

---

## 4. Donor Workflow

Donors are the foundation of pApAmA's mission — their generosity funds every meal served through the platform. The donor workflow is designed to make giving simple, transparent and connected to real humanitarian outcomes.

### 4.1 Making a Donation

Page: `/donor/donate`

The donor signs in at `/donor/login` (or signs up at `/donor/signup`) and navigates to the Donate page.

1. The donor enters the amount to donate (in ₹)
2. The donor chooses a payment method: - **Scan & Pay (UPI QR)** — The donor sees a QR code, scans it with any UPI app (Google Pay, PhonePe, Paytm, etc.) and completes the payment. After paying, the donor enters the **UTR number** (the Unique Transaction Reference — the transaction reference from the UPI app) for manual reconciliation. This is not an integrated payment gateway — the admin reconciles UPI payments against UTR numbers. - **Card, Net Banking, Bank Transfer** — These methods are available for demonstration purposes only and are not connected to a live payment gateway.
3. After payment, the donor sees a confirmation page with their updated credit balance.

**Guest donations:** Anyone can donate without an account at the public `/donate` page. Guest donations go into the General Donation Pool (shown as "Guest Pool" in the application interface), which the admin manages and converts into tokens for volunteer distribution.

### 4.2 Understanding Donor Credit

Page: `/donor/credit`

When a donor donates, the payment becomes **Donor Credit** — a non-withdrawable internal accounting balance within pApAmA. It is not a wallet and cannot be transferred, cashed out or used for anything other than minting meal tokens.

**Credit lifecycle:**

- Credit increases with each donation
- Credit decreases when the donor mints a token
- The donor can view their full credit history (top-ups and deductions)

**Donor Credit does not expire.** This is Trust policy, unless required by future statutory or regulatory provisions. The donor's contribution is a commitment to fund meals for people in need — once donated, funds cannot be withdrawn as cash.

### 4.3 Minting a Token

Page: `/donor/tokens`

Once a donor's credit balance reaches the `standard_token_value` (set by the admin), the donor can mint a token — a digital meal voucher.

**What a token represents:** A token is a one-time entitlement to a freshly prepared meal at any approved Food Partner. Per the approved model, it is PAN INDIA by default — redeemable at any authorised pApAmA Food Partner anywhere in India. It carries a rupee value, a QR code, an activation date and a 60-day expiry date. The activation date is mode-dependent (confirmed 18 Aug): **Donor Controlled** tokens activate at creation; **PAPAMA Distributed** tokens activate when actually distributed/assigned to the beneficiary.

**Built — September 2026 (work order A-2, B-23):** Donor-selected geographic restriction (PAN INDIA / State / District / City / PIN) at token creation; token face content showing type, value, geographic eligibility, activation date and expiry date. Per-mode activation-date rule implementation.



No reminder is sent to anyone before a token expires, and the expire sweep is triggered manually by an administrator rather than running automatically.

#### How to mint:

1. Go to the Tokens page
2. Click **Create Token**
3. Choose the token amount (must be at least the standard token value and cannot exceed the credit balance)
4. Choose the distribution path: - "I'll distribute it myself" (**Path A — Donor Controlled**) — The token goes **live** immediately. The donor gets a QR code to share. This suits donors who know a specific person they want to help. - "Let pApAmA distribute" (**Path B — PAPAMA Distributed**) — The token goes into the **Admin Pool** (**in\_admin\_pool**). The admin allocates it to a volunteer for field distribution. This suits donors who want to fund meals but do not have a specific recipient in mind.
5. The donor's credit balance is reduced by the token amount.

The distribution mode is fixed at creation and cannot be changed afterwards. Donor-controlled tokens never enter the FIFO allocation queue.

## 4.4 Distributing a Token (Path A — Donor Controlled)

When a donor chooses Path A:

1. The token is **live** with a unique QR code
2. The donor views the QR code at `/donor/tokens/[id]`
3. The donor shares the QR code with someone in need — by showing it on screen, printing it, or sharing it digitally
4. The recipient takes the QR code to any approved Food Partner to receive a meal

#### Responsible distribution and transferability caution:

- A Path A token can be redeemed by **whoever presents the QR code first** — there is no binding to a named recipient. A live QR code is a live meal entitlement. The donor should share the QR code discreetly and only with the intended person.
- Do not post token QR codes publicly (social media, public websites, open messaging groups). A publicly posted QR can be redeemed by anyone who scans it first.
- Beneficiaries should never be photographed or publicised in connection with receiving a token.

## 4.5 Letting pApAmA Distribute (Path B — PAPAMA Distributed)

When a donor chooses Path B:

1. The token enters the **Admin Pool** — the central holding area for tokens awaiting distribution
2. The admin allocates the token to a volunteer
3. The volunteer distributes the token (with QR code) to a beneficiary in the field
4. The donor receives a notification when the token is redeemed for a meal

**Built — September 2026 (work order A-4 and F-5, B-32):** FIFO allocation from the pool, ensuring fair sequential distribution.

The **Admin Pool** ensures end-to-end transparency: every token is tracked from the pool, through a named volunteer, to distribution and redemption. Allocation and utilisation are separate events — the donor is notified on actual redemption, not on allocation.

## 4.6 Tracking Impact

Pages: `/donor/dashboard` and `/donor/impact`

The donor dashboard provides an impact summary:

- **Total donated** (₹) — lifetime contributions

- **Credit balance** — funds available for minting tokens
- **Tokens minted** — how many tokens the donor has created
- **Meals served** — how many of the donor's tokens have been redeemed for meals
- **Monthly trends** — donation and impact history by month

This framing connects a financial contribution to a humanitarian outcome: a donation funded a meal, and the meal was served to a person in need.

## 4.7 Notifications

Page: `/donor/notifications`

Donors receive **in-app notifications only**. SMS, email and WhatsApp notification channels are not active — adapter stubs exist but skip delivery when provider API keys are not configured. No multi-language support exists.

### Approved notification templates:

Per the approved Beneficiary Privacy and Donor Communication policy, donor-facing communications contain only information necessary to confirm donation/token utilisation and communicate programme impact. The following are the standard templates — the only donor-visible redemption content:

| TOKEN TYPE   | TEMPLATE                                                                                                                                                                           |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Standard     | PAPAMA Redemption Update — Your sponsored meal token has been successfully redeemed in [City, State]. Thank you for helping PAPAMA provide food with dignity.                      |
| Special Care | PAPAMA Special Care Update — Your ₹100 Special Care Token has been successfully redeemed in [City, State]. Thank you for supporting PAPAMA's Special Care programme.               |
| Emergency    | PAPAMA Emergency Response Update — Your sponsored emergency meal token has been successfully redeemed in [City, State]. Thank you for supporting PAPAMA's emergency food response. |

The location follows the **City + State actual-location convention** — e.g., a Coimbatore-sponsored unrestricted token redeemed in Mumbai reads "Mumbai, Maharashtra".

### Current notification contents (to be aligned):

The current implementation sends two notifications per redemption with metadata including:

`vendor_name`, `meal_info` (menu item name), `location` (vendor city), `time`, `value_inr`, `beneficiary_category` and `token_reference`. The `beneficiary_category` is currently included in notification metadata and is visible to the donor.

Per the approved policy, beneficiary category shall **not** be disclosed to donors — it is sensitive information restricted to authorised pApAmA personnel on a need-to-know basis.

**Built — September 2026 (work order P-1, P-2 and P-3, B-29):** Notification engine whitelist filter — sensitive fields (including beneficiary category, health status, vulnerability information) structurally unreachable from donor-facing communications. Need-to-know role access tiers. Adoption of the three approved templates above as the standard notification content.

**Template editing:** Notification message templates are editable by an administrator at `/admin/notification-templates` (see Section 3.14). Templates use placeholders such as `{{token_value}}` and `{{vendor_name}}`.

**Persistence:** Notification history persists indefinitely. Notifications are marked as read when viewed but are never deleted.

## 4.8 Reporting a Lost Token (Donor)

Donors can self-service report a lost token through the API at `POST /api/donor/tokens/[id]/report-loss`. This triggers the same lost-token replacement process described in Section 3.3 — the old token is blocked and a new replacement is minted with the same value, linked via `replacement_for_token_id`.



## 5. Food Partner Workflow

Food Partners (shown as "Vendor" in the application interface) are humanitarian partners of the pApAmA Trust. They prepare and serve fresh meals to beneficiaries, collect the beneficiary contribution on behalf of pApAmA, upload proof of service and receive payment through settlements. This section describes their complete workflow.

### 5.1 Registering as a Food Partner

Page: `/vendor/register`

Registration and approval journey:

1. Go to `/vendor/register`
2. Fill in business details: - Shop/restaurant name - Address and geo-location (coordinates) — these coordinates are used for the redemption radius check (`token_redemption_radius_km`), so they should be the establishment's actual position rather than an approximate area - Phone number and email - FSSAI licence number (no expiry date is captured) - GST number (if applicable) - Emergency contact
3. Upload required KYC documents — these are stored in a private storage bucket and served to admins via short-lived signed URLs. They are not publicly accessible.
4. Submit the application

The registration enters `pending` status. An administrator or vendor manager reviews the application and KYC documents, then approves or rejects it. Once approved, the Food Partner can sign in at `/vendor/login`.

**Built — September 2026 (work order A-1, B-02):** Structured geographic address fields — State/District masters, City/Town/Village/Locality, 6-digit PIN validation, location IDs, and registered-vs-operating/service address maintained separately. Current build uses city string + coordinates.

### 5.2 Managing the Menu

Page: `/vendor/menu`

After approval, the Food Partner sets up their menu:

1. Go to **Menu**
2. Click **Add Item** — enter the item name, description and price (in ₹)
3. Mark the item as a Special Care equivalent if applicable (`is_special_care_equivalent`)
4. Submit for admin approval
5. Once approved, the item is available for meal redemptions

**Special Care categories as implemented:** Special Care items serve beneficiaries in the `pregnant_women` and `patient` categories. The `special_care_multiplier` config key exists in the system but is an **internal analysis parameter only** — it is defined but never applied in any token minting, redemption or value calculation code. It is maintained within an approved range of approximately 1.5x to 2x of the standard token value for financial and policy analysis purposes, but the donor-facing Special Care Token value is fixed at ₹100 per the approved Special Care programme.

**Pricing:** Menu price edits overwrite the previous value — no price history is maintained. There is no per-item availability toggle; the Food Partner manages availability at the establishment level (see Section 5.3).

### 5.3 Availability and Capacity Management

Page: `/vendor/availability`

The Food Partner manages their operational status through three controls:

| CONTROL                              | PURPOSE                                                    | EFFECT ON REDEMPTIONS                                                                                                              |
|--------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>is_open</code>                 | General open/closed toggle                                 | When closed ( <code>false</code> ), all redemptions are <b>hard-blocked</b>                                                        |
| <code>temporary_closure_until</code> | Scheduled temporary closure with a return date/time        | Redemptions are <b>hard-blocked</b> until the specified time passes. Shown as "Temporarily closed" on the beneficiary nearby list. |
| <code>stock_exhausted</code>         | Indicates the Food Partner has run out of food for the day | Redemptions are <b>hard-blocked</b> until reset                                                                                    |

**Daily capacity:** The Food Partner sets a `daily_meal_capacity` — the maximum meals they can serve per day. When `vendor_capacity_enforcement_enabled` is ON in System Config, redemptions stop (hard block) when the Food Partner reaches their daily limit. Capacity is not displayed on the beneficiary-facing nearby list — only the admin console shows remaining capacity via `/admin/vendor-capacity`.

**Planned (B-13):** Beneficiary-facing redirection to nearby open Food Partners when capacity or closure blocks a redemption. Currently, the redemption simply fails with an appropriate message.

**Meal session participation:** The Food Partner's serving windows (breakfast, lunch, dinner, snack) are configured by the admin at `/admin/meal-windows`. When `meal_window_enforcement_enabled` is ON, redemptions are only accepted during active windows.

## 5.4 Redeeming a Token (Serving a Meal)

Page: `/vendor/scan`

This is the core Food Partner workflow — serving a meal to someone holding a pApAmA token.

**Redemption philosophy:** Every token holder is entitled to a meal. The Food Partner serves all token holders with equal dignity, regardless of category, appearance or circumstance. The Food Partner selects the menu item served from their approved menu — the beneficiary does not choose.

**Step-by-step:**

1. Go to the Scan page at `/vendor/scan`
2. Scan the token QR code — use the phone camera, or paste the code manually (a paste fallback exists for when the camera cannot read the QR)
3. Select the menu item being served — the Food Partner chooses from their approved menu
4. Capture the beneficiary's face — the camera captures a face for identity verification. This is mandatory; there is no manual fallback to skip face capture. Only a mathematical embedding is computed on-device and transmitted — no photograph is stored. No offline mode exists.
5. Collect the ₹10 beneficiary contribution — per the approved contribution policy: - The ₹10 is a beneficiary contribution to pApAmA, collected by the Food Partner as an authorised collection agent - If the beneficiary is unable to contribute, an authorised waiver is applied and recorded - No beneficiary is ever denied food for inability to pay the ₹10 - The contribution is recorded against the redemption transaction
6. Confirm the redemption

**Current implementation note:** The admin can set `co_contribution_max` to any non-negative value (no code-enforced ceiling). The vendor scan UI currently hardcodes a ₹5 maximum (`CO_PAY_MAX = 5` in `app/vendor/scan/page.tsx`).

**Built — September 2026 (work order Q-1, B-10):** Enforce a hard upper bound of ₹10 on `co_contribution_max` in the config validation route.

**Built — September 2026 (work order Q-2, B-20):** Align the client-side `CO_PAY_MAX` in the vendor scan UI with the server-side `co_contribution_max` configuration value.

**₹10 contribution settlement treatment:**

The ₹10 beneficiary contribution and the Food Partner's meal settlement are completely separate:

- **Food Partner receives:** Full approved meal value (`min(token_value, menu_value)`) via settlement
- **₹10 contribution:** Collected at counter, retained by Food Partner, remitted to pApAmA Administration Account under the CA-approved process
- **Settlement payout formula:** `min(token_value, menu_value)` — the ₹10 contribution is entirely excluded from this calculation

**Built — September 2026 (work order F-1, B-01):** ₹10 contribution enforcement system — contribution status per redemption (collected/waived/outstanding), waiver records, daily remittance/reconciliation records, settlement-release gate on contribution reconciliation, and contribution report (expected/collected/remitted/received/outstanding/waived/settled).

**System checks (all must pass):**

- Is the QR code valid and not already used?
- Is the Food Partner open (`is_open`), not stock-exhausted, and not temporarily closed?
- Is the Food Partner within daily capacity (if enforced)?
- Is it within a valid meal window (if enforced)?
- Is the beneficiary within the geofence radius (`token_redemption_radius_km`)?
- Has the beneficiary's cooldown period elapsed?
- Has the beneficiary's daily meal limit not been exceeded?
- Is the token still valid (not expired, blocked or already redeemed)?

**Accessibility:** No smartphone is required of the beneficiary. A printed QR code works. However, face capture at the point of service is mandatory and requires a camera-equipped device on the Food Partner's side.

**Non-discrimination:** The Food Partner must serve all token holders equally. The platform does not display the beneficiary's category to the Food Partner at the point of service.

## 5.5 Uploading Proof of Service

Page: `/vendor/redemptions`

After serving a meal, the Food Partner must upload proof before payment is released.

1. Go to **Redemptions**
2. Find the meal just served (status: "Proof Needed")
3. Upload a **plate photo** — a clear photo of the meal served
4. Submit the proof

**Photo quality guidance:**

- The plate photo should clearly show the food served
- The photo should show the meal, not the beneficiary
- Photos should be clear and well-lit

**What happens next:**

- The admin reviews the proof at `/admin/proofs`
- If **approved** — payment is released and included in the next settlement
- If **rejected** — the Food Partner sees the rejection reason (mandatory for the reviewer). The Food Partner can re-upload improved proof. No resubmission deadline exists.

**Important:** Proof photos are write-once. Once uploaded, they cannot be altered or deleted by the Food Partner. This ensures the integrity of the proof record.

**Mandatory reason:** Proof rejection is the only action in the platform that enforces a mandatory reason at the server level. Other actions (donation reversal, token revocation, vendor suspension, settlement holds) accept an optional reason.

## 5.6 Viewing Settlements

Page: </vendor/settlements>

The Food Partner has a read-only view of their payment status:

| STATUS     | MEANING                                               |
|------------|-------------------------------------------------------|
| Pending    | Approved meals waiting for the next settlement cycle  |
| Locked     | Settlement prepared and locked for review             |
| Approved   | Reviewed and approved for reconciliation              |
| Reconciled | Pre-payment reconciliation complete, awaiting payment |
| Paid       | Funds transferred to the Food Partner's account       |

The admin controls when settlements are run and payments are released. There is no Food Partner-side dispute or query mechanism for settlements.

## 6. Volunteer Workflow

Volunteers are the bridge between the platform and beneficiaries in the field. They carry tokens from the Admin Pool to people in need — a role of trust and accountability. The volunteer's role is to facilitate access to pApAmA assistance, not to create or alter entitlement.

### 6.1 Registering as a Volunteer

Page: </volunteer/register>

Registration and verification journey:

1. Enter name, phone number and email
2. Capture your face (for identity verification) — a mathematical embedding is computed on-device; only the embedding is stored, no photograph is retained
3. Submit the registration

The volunteer's email is pre-confirmed server-side (no email verification step). The registration enters `pending` status and must be approved by an administrator. After approval, the volunteer signs in at </volunteer/login>.

**Volunteer responsibilities:**

- Distribute tokens to beneficiaries promptly and equitably
- Assist beneficiaries with registration when needed (at </volunteer/beneficiaries>)
- Maintain accurate records of distribution activity
- Never demand payment for a token
- Protect beneficiary privacy — no photographs or public identification

### 6.2 Receiving and Managing Tokens

Page: </volunteer> (Dashboard)

**Purpose of allocation:** Token allocation connects donor generosity to beneficiary need through a named, accountable volunteer. Every token passing through a volunteer is tracked — from pool allocation through distribution to redemption.

**Two ways to receive tokens:**

**Option 1 — Admin assigns tokens:**

- The admin selects the volunteer's profile and allocates tokens from the Admin Pool
- Tokens appear in the volunteer's "Held Tokens" section

**Option 2 — Volunteer requests tokens:**

1. Go to the **Request Tokens** section on the dashboard
2. Enter how many tokens are needed
3. Submit the request
4. The admin reviews and approves (fully, partially, or denies)
5. Approved tokens appear in the "Held Tokens" section

**Holding limits:** The volunteer can only hold a set number of undistributed tokens at a time (`max_tokens_per_volunteer`). This limit exists to bound risk — if a volunteer becomes unreachable, the maximum number of tokens at risk is capped. The dashboard shows the current count and remaining capacity.

**Token accountability:** A volunteer cannot return an undistributed token to the Admin Pool. Only an admin can revoke a held token back to the pool. This ensures that every token movement is an explicit administrative action with an audit trail.

### 6.3 Distributing Tokens to Beneficiaries

Page: </volunteer> (Dashboard → Held Tokens)



**Purpose and dignity:** Volunteers are not distributing QR codes — they are carrying donor-funded meals to people in need, with dignity, compassion and fairness. Every interaction should preserve the beneficiary's self-respect. Avoid anything that could embarrass or stigmatise a person receiving a token.

**Governing principle:** Volunteers shall facilitate beneficiary access to pApAmA assistance but shall not create, modify or independently authorise beneficiary entitlement.

**Seven field situations:**

**Situation 1 — Beneficiary has a smartphone:** The beneficiary receives a digital QR code. Normal flow — the volunteer distributes the token, the beneficiary presents the QR at a Food Partner, and the transaction proceeds through standard validation.

**Situation 2 — No smartphone, but has a printed QR:** The volunteer presents the official pApAmA printed QR token on behalf of the beneficiary. Normal validation applies — the Food Partner scans the printed QR and proceeds with standard verification.

**Situation 3 — No phone and no token, during normal operations:** The volunteer follows the approved assistance and registration process. If the person is not yet registered, the volunteer assists with registration at [/volunteer/beneficiaries](#). If already registered but without a token, the volunteer facilitates access through normal administrative channels (token allocation from the admin). The volunteer does not improvise or create unverified transactions.

**Situation 4 — No phone or token during Emergency Mode:** During an authorised Emergency Mode, relaxed verification applies. The volunteer assists the person to the nearest active Food Partner and facilitates an emergency redemption with the available controls. The transaction is recorded with Emergency Mode tagging.

**Built — September 2026 (work order E-2 and E-3, B-27):** Relaxed beneficiary verification during Emergency Mode with verification level recording. ₹10 waiver applied to emergency transactions where authorised.

**Situation 5 — No connectivity during normal operations:** The volunteer does not improvise. There is no offline mode during normal operations. The volunteer should:

- Direct the person to a connected Food Partner if one is nearby
- Submit an assistance request through administrative channels
- Escalate to an administrator

Both distribution and redemption require connectivity. Face capture at redemption has no fallback. This is a real operational constraint.

**Situation 6 — No connectivity during Emergency Mode:** During an authorised Emergency Mode, controlled offline emergency transactions may be permitted. **Two capture routes are confirmed (18 Aug):**

| ROUTE                          | ROLE      | USAGE                                                                           |
|--------------------------------|-----------|---------------------------------------------------------------------------------|
| Food Partner redemption screen | Primary   | The Food Partner's scan page records the offline transaction at their premises. |
| Volunteer App                  | Secondary | For field situations where no connected Food Partner is available.              |

Both routes are emergency-only, subject to identical controls. **Every offline transaction must record its source** (Food Partner or Volunteer) as a mandatory field, enabling separate post-emergency review by source.

**Built — September 2026 (work order E-4, B-30):** Controlled offline emergency transaction capability — offline transaction record (offline txn ID, token ID, volunteer ID, Food Partner ID, beneficiary identifier, Emergency ID, date/time, token/meal type, waiver status, **transaction source** (Food Partner / Volunteer), device reference), synchronisation with full normal validation ("Pending Offline Validation"), configured limits (max pending per volunteer/device, max sync window), cross-batch duplicate-token detection with exception-queue routing, and admin visibility of unsynchronised transactions. This is emergency-only — normal operations have no offline path.

**Situation 7 — Immediate safety risk:** Safety first. If the volunteer encounters an unsafe situation, they should prioritise their own safety and the beneficiary's safety. Escalate to the administrator. Never force a transaction in an unsafe environment. Volunteer safety and food-safety requirements apply at all times.

**Built — September 2026 (work order E-5, B-31):** Volunteer incident reporting — 11 one-tap report categories (no phone / no token / partner closed / partner refusing valid token / no food / connectivity failure / token problem / urgent need / food-safety concern / safety concern / other) feeding an admin queue.

**Volunteers shall not:**

- Independently activate Emergency Mode
- Alter token value or expiry
- Override geographic restrictions
- Reactivate suspended Food Partners
- Bypass pApAmA controls

**Distribution steps (standard):**

1. Find the token you want to distribute in your "Held Tokens" list
2. Click **Distribute**
3. The QR code is displayed — show it to the beneficiary or share it digitally
4. The token status changes from `assigned_to_volunteer` to `distributed`
5. Your holding count decreases, freeing up space for more tokens

**Accountability:** The `assigned_to_volunteer` → `distributed` transition is recorded with a timestamp and distribution channel. This means the token remains traceable through its whole life — from donor credit, through minting, pool allocation, volunteer holding, distribution and finally redemption at a Food Partner.

**Registration assistance:** Volunteers can assist beneficiaries with registration at `/volunteer/beneficiaries` by helping with data entry. The volunteer-assisted registration form collects the same five fields as self-registration (category, name, contact, location hint and face capture). Approval authority remains with the Trust through the normal administrative review process.

**Digital accessibility:** A token can be shown on the volunteer's phone screen, shared electronically, or printed as a QR code. The beneficiary does not need a smartphone — a printed QR code is the normal case, not an exception.

**Distribution records:** The "Distributed" section on the volunteer dashboard shows previously distributed tokens with serial number, token type, value (₹) and current status (e.g. `distributed`, `redeemed`, `expired`). Distribution date is not displayed in the list. Distribution location is captured when the volunteer submits a distribution but is not shown back in the distributed list.

Distribution-record enrichment (date, value, location, redemption/expiry status in the list) and a Volunteer Field Activity Dashboard are planned for a future phase (Phase 2 backlog).

## 7. For Beneficiaries — Receiving Meals

pApAmA exists to serve beneficiaries — people in need of a meal. Every other role on the platform exists to make this service possible. The beneficiary's experience is designed to be simple, dignified and barrier-free.

### 7.1 Registering as a Beneficiary

Page: `/beneficiary/register`

Anyone in need can register. No Aadhaar, smartphone, bank account or formal ID is mandatory.

1. Go to `/beneficiary/register` (or ask a volunteer for help)
2. Enter your name and basic details
3. Select your category: - Pregnant Women - Patients - Persons with Disabilities - Disaster-Affected (during emergencies)
4. Capture your face — a mathematical embedding is computed on-device for identity verification at meal time. No photograph is ever transmitted to or stored on the server; only the non-reversible embedding is retained.
5. Submit

An admin will review and approve your registration. The approval decision is based on category, face verification and administrative judgement.

**Note:** These four categories are hardcoded in the current implementation (`lib/types/enums.ts`). Adding a category requires a code change.

**Future categories:** Children, Elderly Persons, Lactating Mothers and General Category are planned for a future phase.

### 7.2 Finding a Nearby Food Partner

Page: `/beneficiary/nearby-vendors`

This page helps beneficiaries find an approved pApAmA Food Partner (shown as "Vendor" in the application interface) near their location.

**How it works:**

- The beneficiary clicks "Use my location" to share their browser GPS position (on-demand, not stored)
- The system computes the distance to each Food Partner using Haversine (great-circle) distance
- Results are listed within the configured radius (`token_redemption_radius_km`)

**What each listing shows:**

| FIELD                                                      | SHOWN                         |
|------------------------------------------------------------|-------------------------------|
| Food Partner name                                          | Yes                           |
| Address / city                                             | Yes                           |
| Distance (km)                                              | Yes                           |
| Status (Open / Closed / Temporarily closed / Out of stock) | Yes                           |
| Operating hours (meal windows)                             | Yes                           |
| Contact number                                             | No                            |
| Menu items                                                 | No (separate menu API exists) |

### 7.3 Redeeming a Token at a Food Partner

Once you have a token (received from a donor or volunteer):

1. Visit any approved pApAmA Food Partner
2. Show your token QR code to the Food Partner (on your phone or printed)
3. The Food Partner scans the QR code, selects a menu item from their approved menu and verifies your face
4. **Contribute ₹10** — a small contribution to pApAmA, collected by the Food Partner. If you cannot pay, the contribution is waived — no beneficiary is ever denied a meal for inability to pay.
5. You receive your meal — freshly cooked, served at the premises
6. That's it! The token is consumed and the Food Partner handles the rest

#### Meal limits (for fairness):

- There is a minimum waiting period between meals (cooldown, e.g. 6 hours)
- There is a maximum number of meals per day
- Category-specific cooldown overrides may apply (e.g. relaxed cooldown for pregnant women or patients)

#### Exception behaviours:

| EXCEPTION                     | WHAT HAPPENS                                                            | MESSAGE SHOWN                                                                                                            |
|-------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Expired token                 | Hard block — redemption refused                                         | "token has expired"                                                                                                      |
| Already-redeemed token        | Hard block — duplicate scan detection rolls back redemption             | "token was already redeemed"                                                                                             |
| Failed face verification      | Hard block — identity check failed                                      | "could not verify beneficiary identity" or "face capture failed the liveness/anti-spoof check — retake in good lighting" |
| Network interruption          | Graceful failure — no offline queue; token not burned until full commit | "Network error — please try again."                                                                                      |
| Food Partner capacity reached | Hard block (if <code>vendor_capacity_enforcement_enabled</code> is ON)  | "vendor daily capacity reached (served/cap)"                                                                             |
| Outside meal window           | Hard block (if <code>meal_window_enforcement_enabled</code> is ON)      | "outside meal windows — next window opens at [time]"                                                                     |

**Face privacy:** Only a non-reversible 1024-dimension face embedding is retained. The face image is computed on-device and never leaves the device — no photograph is ever transmitted to or stored on the server.

## 7.4 Giving Feedback

Page: `/beneficiary/feedback`

After a meal, you can share your experience:

1. Go to `/beneficiary/feedback`
2. Select the Food Partner
3. Rate your experience (1–5 stars)
4. Add comments about food quality, quantity, or service (optional, up to 1000 characters)
5. Check the complaint box if this is a serious issue ("This is a complaint — hygiene, behaviour, or a serious problem")
6. Submit

Feedback is a star rating plus free-text comment plus a binary `is_complaint` flag. There are no predefined complaint categories in the current implementation.

**Planned (B-12):** Predefined complaint categories (quality, quantity, hygiene, staff behaviour, delay, redemption issue, other) to enable structured triage and root-cause reporting.

Complaints enter a status workflow: **Open** → **Investigating** → **Resolved / Dismissed**. Resolution is recorded with notes and the resolving administrator.

Your feedback helps the admin monitor Food Partner quality and is factored into the Food Partner's quality score.

---

## 8. Public Features (No Account Needed)

### 8.1 Public Donation — General Donation Pool

Page: `/donate`

Anyone can donate without creating an account. The donation goes into the **General Donation Pool** (shown as "Guest Pool" in the application interface) — a dedicated fund managed by the admin for converting into tokens and distributing to beneficiaries through volunteers.

The General Donation Pool operates with the same governance and audit standards as registered donor contributions. Every rupee is tracked from donation through token conversion, volunteer allocation, distribution, redemption and settlement.

No contact information (mobile or email) is captured from guest donors. Even if notification channels were configured, there is no contact data to deliver to.

**Planned (B-11):** Capture optional contact (mobile/email) on guest donation for receipt and thank-you notifications.

### 8.2 UPI QR Donation

Page: `/donate/qr`

Scan a UPI QR code with any UPI app (Google Pay, PhonePe, Paytm, etc.) and confirm the payment with your UTR number.

**UTR (Unique Transaction Reference):** The UTR is a self-asserted transaction reference entered by the donor. It is used for manual reconciliation — the admin matches UTR numbers against bank records. The UTR is **not** verified against any bank or payment service provider feed.

**Duplicate UTR protection:** A database uniqueness constraint prevents the same UTR from being used twice. If a duplicate UTR is submitted, the system returns HTTP 409: "this UPI reference number has already been used to confirm a payment."

**Optional `payer_vpa` capture:** The UTR confirmation endpoint also accepts an optional `payerVpa` field — the donor's UPI virtual payment address. This is captured for reconciliation support but is not required.

| SCENARIO                 | BEHAVIOUR                                                              |
|--------------------------|------------------------------------------------------------------------|
| Payment failure          | Manual re-entry (user can try again)                                   |
| Incorrect UTR format     | Client-side validation: min 6, max 40 characters                       |
| Pending status (timeout) | Lazy expiry flip to <code>EXPIRED</code> on first touch after deadline |
| Duplicate UTR            | First confirms atomically; second gets 409 Conflict                    |

**Planned (B-21):** UTR verification against bank/PSP feed (currently donor-self-asserted; a fabricated UTR can mint pool credit). This should ride the live payment gateway integration workstream.

### 8.3 Transparency Dashboard

Page: `/transparency`

A public page showing aggregate platform impact — total donations, meals served, Food Partners, and beneficiaries reached. No personal information is ever shown. All figures derive from verified platform records.

This page is only visible when `transparency_dashboard_enabled` is turned ON by the admin.

## 9. System Configuration Reference

These settings are managed by an authorised administrator at `/admin/system-config`. Each setting controls a specific platform rule.

### Configuration Governance

- **Access:** Only authorised administrators may change system configuration values.
- **Audit trail:** Every change is audit-logged with the previous value, new value, actor (`actor_id`) and timestamp. The audit action is `system_config.update` with summary format `"key: old → new"`.
- **Built — September 2026 (work order Q-3, B-14):** Every configuration change carries a reason in the audit trail, recorded beside who changed it, when, and the before and after values. A reason is mandatory when extending an active emergency override.
- **Approval practice:** Significant changes (token value, contribution policy, emergency activation, audit rate changes) should follow the Foundation's internal approval practice before being applied in the system.
- **Permanent rate reductions:** The settlement audit rate should only be permanently reduced after review with the accounting/audit advisor.

### Classification key

| TAG                   | MEANING                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------|
| <b>Mandatory</b>      | Must be set before go-live; system behaviour is undefined or degraded without it            |
| <b>Recommended</b>    | Should be set for proper operation; system functions without it but with reduced governance |
| <b>Optional</b>       | Enable as needed; system operates correctly without it                                      |
| <b>NULL-permitted</b> | NULL has a specific, stated meaning (not a universal "soft-skip")                           |

### 9.1 Token Settings

| KEY                                   | TYPE       | CLASSIFICATION   | WHAT IT CONTROLS                                                                                                                                               | NULL MEANING                          | DEFAULT     | BUSINESS IMPLICATION                                                     |
|---------------------------------------|------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|-------------|--------------------------------------------------------------------------|
| <code>standard_token_value</code>     | Number (₹) | <b>Mandatory</b> | Minimum and pool-mint token value                                                                                                                              | Tokens cannot be minted               | Must be set | Determines the rupee value of every standard meal token                  |
| <code>token_expiry_days</code>        | Number     | <b>Mandatory</b> | Days before an unused token expires from activation (activation = creation for Donor Controlled; distribution event for PAPAMA Distributed — confirmed 18 Aug) | No expiry — tokens live indefinitely  | NULL        | Set to <b>60</b> per approved policy. Unset = unbounded token liability. |
| <code>max_tokens_per_volunteer</code> | Number     | <b>Mandatory</b> | Maximum undistributed tokens a volunteer can hold at once                                                                                                      | No holding limit — exposure unbounded | NULL        | Caps risk if a volunteer becomes unreachable                             |

### 9.2 Meal and Redemption Settings

| KEY                              | TYPE   | CLASSIFICATION   | WHAT IT CONTROLS                              | NULL MEANING                           | DEFAULT     | BUSINESS IMPLICATION              |
|----------------------------------|--------|------------------|-----------------------------------------------|----------------------------------------|-------------|-----------------------------------|
| <code>meal_cooldown_hours</code> | Number | <b>Mandatory</b> | Minimum hours between meals for a beneficiary | No cooldown — unlimited meal frequency | Must be set | Prevents rapid repeat redemptions |

| KEY                                                | TYPE       | CLASSIFICATION | WHAT IT CONTROLS                                                          | NULL MEANING                                  | DEFAULT     | BUSINESS IMPLICATION                                                        |
|----------------------------------------------------|------------|----------------|---------------------------------------------------------------------------|-----------------------------------------------|-------------|-----------------------------------------------------------------------------|
| <code>meal_cooldown_hours_pregnant_women</code>    | Number     | Optional       | Relaxed cooldown for pregnant women                                       | Uses general <code>meal_cooldown_hours</code> | NULL        | Category-specific relaxation                                                |
| <code>meal_cooldown_hours_patient</code>           | Number     | Optional       | Relaxed cooldown for patients                                             | Uses general <code>meal_cooldown_hours</code> | NULL        | Category-specific relaxation                                                |
| <code>meal_cooldown_hours_disability</code>        | Number     | Optional       | Relaxed cooldown for persons with disabilities                            | Uses general <code>meal_cooldown_hours</code> | NULL        | Category-specific relaxation                                                |
| <code>meal_cooldown_hours_disaster_affected</code> | Number     | Optional       | Relaxed cooldown for disaster-affected                                    | Uses general <code>meal_cooldown_hours</code> | NULL        | Category-specific relaxation                                                |
| <code>max_meals_per_day</code>                     | Number     | Mandatory      | Maximum meals a beneficiary can receive per day                           | No daily limit                                | Must be set | Ensures equitable distribution                                              |
| <code>token_redemption_radius_km</code>            | Number     | Mandatory      | Maximum distance (km) between Food Partner and beneficiary for redemption | No distance check                             | Must be set | Prevents remote/proxy redemptions                                           |
| <code>meal_window_enforcement_enabled</code>       | Boolean    | Recommended    | Block redemptions outside defined meal windows                            | —                                             | OFF         | Enable with configured windows at <code>/admin/meal-windows</code>          |
| <code>co_contribution_max</code>                   | Number (₹) | Mandatory      | Maximum beneficiary contribution at redemption                            | Only ₹0 accepted                              | NULL        | Set to 10 (₹10 approved policy). No code-enforced ceiling currently exists. |

### 9.3 Food Partner Settings

| KEY                                      | TYPE         | CLASSIFICATION | WHAT IT CONTROLS                                              | NULL MEANING                                  | DEFAULT | BUSINESS IMPLICATION                                                                       |
|------------------------------------------|--------------|----------------|---------------------------------------------------------------|-----------------------------------------------|---------|--------------------------------------------------------------------------------------------|
| <code>vendor_auto_suspend_enabled</code> | Boolean      | Optional       | Auto-flag Food Partners when complaint rate exceeds threshold | —                                             | OFF     | The graduated corrective-action ladder governs operationally; this remains OFF             |
| <code>vendor_max_complaint_rate</code>   | Number (0–1) | Recommended    | Complaint ratio threshold for auto-suspend flagging           | Auto-suspend never triggers (even if enabled) | NULL    | Set in context — a single complaint from few interactions may not indicate systemic issues |
| <code>vendor_min_rating</code>           | Number       | Optional       | Minimum acceptable Food Partner rating                        | No minimum enforced                           | NULL    | Quality floor — rating is one signal among several                                         |



| KEY                                              | TYPE    | CLASSIFICATION     | WHAT IT CONTROLS                                              | NULL MEANING                             | DEFAULT | BUSINESS IMPLICATION                                                   |
|--------------------------------------------------|---------|--------------------|---------------------------------------------------------------|------------------------------------------|---------|------------------------------------------------------------------------|
| <code>vendor_min_feedback_count</code>           | Number  | <b>Optional</b>    | Minimum feedback entries before rating is considered reliable | Rating always considered                 | NULL    | Prevents premature quality judgements from few data points             |
| <code>vendor_capacity_enforcement_enabled</code> | Boolean | <b>Recommended</b> | Enforce Food Partner daily capacity limits                    | —                                        | OFF     | Enable after Food Partners set <code>daily_meal_capacity</code>        |
| <code>vendor_inspection_fail_penalty</code>      | Number  | <b>Recommended</b> | Quality-score deduction on failed inspection                  | No penalty applied on failed inspections | NULL    | Numeric value deducted from quality score (e.g. 10 = deduct 10 points) |

## 9.4 Settlement and Financial Settings

| KEY                                       | TYPE         | CLASSIFICATION   | WHAT IT CONTROLS                                 | NULL MEANING    | DEFAULT | BUSINESS IMPLICATION                                            |
|-------------------------------------------|--------------|------------------|--------------------------------------------------|-----------------|---------|-----------------------------------------------------------------|
| <code>settlement_random_audit_rate</code> | Number (0–1) | <b>Mandatory</b> | Fraction of settlements sampled for random audit | No random audit | NULL    | Set to <b>0.10</b> (10% baseline). "Baseline, not the ceiling." |

## 9.5 Emergency Mode Settings

| KEY                                           | TYPE    | CLASSIFICATION                       | WHAT IT CONTROLS                                | NULL MEANING                                                    | DEFAULT | BUSINESS IMPLICATION                                                         |
|-----------------------------------------------|---------|--------------------------------------|-------------------------------------------------|-----------------------------------------------------------------|---------|------------------------------------------------------------------------------|
| <code>emergency_mode_enabled</code>           | Boolean | <b>Optional</b>                      | Activate disaster/emergency relief mode         | —                                                               | OFF     | Review all emergency values before activation                                |
| <code>emergency_max_meals_per_day</code>      | Number  | <b>NULL-permitted</b>                | Relaxed daily meal limit during emergency       | No daily cap during emergency                                   | NULL    | Set to <b>4</b> per approved policy                                          |
| <code>emergency_meal_cooldown_hours</code>    | Number  | <b>NULL-permitted</b>                | Relaxed cooldown during emergency               | No cooldown during emergency                                    | NULL    | Set to <b>3</b> per approved policy                                          |
| <code>emergency_mode_max_duration_days</code> | Number  | <b>Mandatory (before activation)</b> | Auto-revert emergency mode after this many days | Never auto-reverts — emergency persists until manually disabled | NULL    | Set to <b>7</b> per approved policy. No indefinite Emergency Mode permitted. |

## 9.6 Location Settings

| KEY                            | TYPE    | CLASSIFICATION                     | WHAT IT CONTROLS                         | NULL MEANING            | DEFAULT | BUSINESS IMPLICATION                                                                |
|--------------------------------|---------|------------------------------------|------------------------------------------|-------------------------|---------|-------------------------------------------------------------------------------------|
| <code>city_lock_enabled</code> | Boolean | <b>Recommended (pilot)</b>         | Restrict redemptions to a single city    | —                       | OFF     | Pilot-phase operational control. Enforced at redemption only — not at registration. |
| <code>operating_city</code>    | String  | <b>Mandatory (if city lock ON)</b> | The operating city for redemption gating | City lock has no effect | NULL    | Standardise city name for consistent matching                                       |

**City lock enforcement scope:** City lock is enforced **only at token redemption** — the system compares the Food Partner's city against the operating city (case-insensitive) and hard-blocks mismatches. Beneficiary registration, Food Partner onboarding and volunteer registration are **not** gated by city lock.

**Built — October 2026 (work order Q-5, B-15):** City lock now applies at registration as well as at redemption. The client chose a hard block over a warning (7 October 2026): while `city_lock_enabled` is on and `operating_city` is set, a beneficiary, volunteer or Food Partner registering from another city is refused, and the city is a required field. With the lock off, or with no operating city configured, the field is ignored and registration is unrestricted.

**Geographic hierarchy:** The approved geographic structure for Phase 1 is Country → State → District → City/Town/Village/Locality with 6-digit PIN validation and location IDs. Current build uses city string + coordinates.

**Built — September 2026 (work order A-1, B-02):** Structured geographic hierarchy with State/District masters, location IDs, per-stakeholder requirement levels, and actual service-location snapshot per redemption transaction.

City lock is a pilot-phase operational control. The token-level geographic restriction model (PAN INDIA / State / District / City / PIN per D-2A) will supersede it.

## 9.7 Quality and Security Settings

| KEY                                   | TYPE   | CLASSIFICATION        | WHAT IT CONTROLS                                                 | NULL MEANING                                   | DEFAULT | BUSINESS IMPLICATION                                                |
|---------------------------------------|--------|-----------------------|------------------------------------------------------------------|------------------------------------------------|---------|---------------------------------------------------------------------|
| <code>proof_phash_dup_distance</code> | Number | <b>Recommended</b>    | How similar two proof photos can be before flagging as duplicate | Duplicate detection OFF                        | NULL    | Lower = stricter matching. Flagged for review, never auto-rejected. |
| <code>audit_log_retention_days</code> | Number | <b>NULL-permitted</b> | How many days to keep audit logs                                 | <b>Permanent retention</b> — logs kept forever | NULL    | Leave NULL. Permanent retention is the approved policy.             |

**Audit log immutability:** Audit logs are append-only. Two PostgreSQL triggers (`audit_logs_no_update`, `audit_logs_no_delete`) call `audit_logs_block_mutation()` which raises an exception for **any** update or delete operation, including by `service_role`. RLS policies permit SELECT for admin/compliance and INSERT-only for authenticated users. No UPDATE or DELETE policies exist. The code exposes only INSERT methods.

**Permanent-retention policy:** Financial, token, settlement and governance audit records are never deleted. The `audit_log_retention_days` key should remain unset (NULL) until a retention policy is formally adopted by the Foundation with appropriate legal and accounting advice.

## 9.8 Feature Toggles

| KEY                                         | TYPE    | CLASSIFICATION  | WHAT IT CONTROLS                                 | NULL MEANING | DEFAULT | BUSINESS IMPLICATION                                                                                                                             |
|---------------------------------------------|---------|-----------------|--------------------------------------------------|--------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>transparency_dashboard_enabled</code> | Boolean | <b>Optional</b> | Show the public <code>/transparency</code> page  | —            | OFF     | Enable when public-facing data is reviewed and approved                                                                                          |
| <code>csr_80g_certificates_enabled</code>   | Boolean | <b>Optional</b> | Enable 80G certificate generation for CSR donors | —            | OFF     | <b>Leave OFF</b> — no generation code exists. Activation requires (a) Foundation 80G registration number and (b) CA-approved certificate format. |
| <code>volunteer_zones_enabled</code>        | Boolean | <b>Optional</b> | Enable volunteer zone geofencing                 | —            | OFF     | Zone assignment is available regardless of this toggle; toggle controls enforcement at distribution                                              |

**Planned (B-16):** 80G certificate generation end-to-end — donor details, transaction reference, Foundation 80G registration, unique certificate number and permanent certificate record. Blocked on external dependencies.

All boolean settings ship **OFF** by default — nothing activates until explicitly configured. Feature toggles follow the same audit trail as other configuration changes.

## 9.9 Special Care Settings

The Special Care programme is designed to provide enhanced nutritional support to beneficiaries with elevated needs. The approved model operates through a separate donor category and fixed-value tokens.

### Approved Special Care programme:

Donors may sponsor Special Care Tokens at **₹100 per token**. Each Special Care Token has a face value of ₹100 and is independently configurable from the standard pApAmA meal token. Where the approved value of the Special Care meal is less than ₹100, the unused balance is automatically credited to the **PAPAMA Common Special Care Pool** — a separate ledger used exclusively for approved Special Care purposes.

**Partly built — September 2026 (work order A-3, B-28; the category master, the ₹100 token value and the Common Special Care Pool were built; the donor sponsorship flow is not yet in place):** Full Special Care programme implementation — SPECIAL\_CARE token type at ₹100, donor sponsorship flow, Common Special Care Pool with separate ledger and automatic surplus routing, configurable Special Care Category Master, and eligibility model with EDD/delivery-date/review-date logic.

### Special Care categories (distinct from beneficiary categories):

The current implementation has four **beneficiary categories** (Pregnant Women, Patients, Persons with Disabilities, Disaster-Affected). The approved Special Care programme introduces a separate **Special Care Category Master** with initial categories:

- Pregnant Women
- Postpartum/Lactating Mothers (new — not in current beneficiary enum)
- Medically Vulnerable/Patients

### Eligibility model (approved policy):

- **Pregnancy:** From verified pregnancy until delivery. Expected delivery date (EDD) recordable. No monthly re-verification required.
- **Postpartum:** Six months from delivery date, auto-calculated.
- **Patients:** No universal period. Per-record category, start date, review/end date, professional recommendation, evidence reference, verification status and verifier. System reminder at review date. The `patient_eligibility_months` configuration key is superseded by this per-record review-date model.

**Medical privacy:** The Food Partner sees only "SPECIAL CARE TOKEN – ₹100" — never the diagnosis, condition or specific Special Care category. Medical details are restricted to authorised pApAmA personnel on a need-to-know basis.

| KEY                                  | TYPE   | CLASSIFICATION         | WHAT IT CONTROLS                                                                                                                                                                                                                     | NULL MEANING | DEFAULT     | BUSINESS IMPLICATION                                                                                                      |
|--------------------------------------|--------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>special_care_multiplier</code> | Number | Internal analysis only | <b>Not functional.</b> Defined in config but never applied in any token minting, redemption or value calculation code. Maintained within an approved range of approximately 1.5x–2x for internal financial and policy analysis only. | Not applied  | Seeded as 2 | Do NOT set with the expectation that it affects token values. The donor-facing Special Care Token value is fixed at ₹100. |

| KEY                                            | TYPE   | CLASSIFICATION | WHAT IT CONTROLS                                                                       | NULL MEANING                             | DEFAULT | BUSINESS IMPLICATION                                                                                                                                                                                                                     |
|------------------------------------------------|--------|----------------|----------------------------------------------------------------------------------------|------------------------------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>special_care_post_delivery_months</code> | Number | Optional       | Months post-delivery a pregnant woman qualifies for Special Care eligibility extension | No post-delivery extension               | NULL    | Sets the eligibility-expiry window for approved pregnant-women beneficiaries                                                                                                                                                             |
| <code>patient_eligibility_months</code>        | Number | Optional       | Sets a universal patient eligibility period (months from approval)                     | Uses general approval without time limit | NULL    | Currently active — used to compute <code>eligibility_expires_at</code> for patients at approval. Will be superseded by per-record review dates with system reminders when the approved Special Care model is implemented (Planned B-28). |

## 9.10 Offline Capture Settings

Offline capture is the platform's one deliberate exception to "the server decides", so its limits are deliberately not guessed. Four of the six keys below are unset, and the feature ships switched off until the client sets them.

| KEY                                             | TYPE    | CLASSIFICATION       | WHAT IT CONTROLS                                                | NULL MEANING       | DEFAULT             | BUSINESS IMPLICATION                                                                                                          |
|-------------------------------------------------|---------|----------------------|-----------------------------------------------------------------|--------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>offline_capture_enabled</code>            | Boolean | Mandatory before use | Whether a counter may record a meal offline during an emergency | Treated as off     | <code>false</code>  | Ships off by design. The offline panel does not appear on the counter screen until this is on AND an emergency is active.     |
| <code>offline_max_pending_per_device</code>     | Number  | Client decision      | How many unsynced captures one device may hold                  | No limit enforced  | NULL                | Unbounded exposure on a single lost or compromised device.                                                                    |
| <code>offline_max_pending_per_volunteer</code>  | Number  | Client decision      | How many unsynced captures one volunteer may hold               | No limit enforced  | NULL                | Unbounded exposure per person rather than per device.                                                                         |
| <code>offline_max_sync_window_hours</code>      | Number  | Client decision      | How long after capture a record may still sync                  | No window enforced | NULL                | A capture of any age is accepted, which weakens the clock check.                                                              |
| <code>offline_unsynced_alert_hours</code>       | Number  | Client decision      | When to alert that a device has not synced                      | No alert raised    | NULL                | A device that stops syncing is noticed late, or not at all.                                                                   |
| <code>offline_rejected_settlement_policy</code> | Text    | Client decision      | What happens to a rejected capture at settlement                | —                  | <code>review</code> | <code>review</code> decides nothing, deliberately. The real choice — pay, withhold, or review case by case — is the client's. |

## 9.11 Verification, Fraud and Value Settings

These keys were in the platform before version 1.2 but were not previously documented here.

| KEY                                             | TYPE   | WHAT IT CONTROLS                                   | DEFAULT | BUSINESS IMPLICATION                                                               |
|-------------------------------------------------|--------|----------------------------------------------------|---------|------------------------------------------------------------------------------------|
| <code>special_care_token_value</code>           | Number | The fixed face value of a Special Care token (₹)   | 100     | The donor-facing Special Care value. Changing it changes what a sponsor is buying. |
| <code>contribution_remittance_cycle_days</code> | Number | How often collected ₹10 contributions are remitted | 1       | Sets the rhythm of the contribution report in Section 3.15.                        |

| KEY                                          | TYPE   | WHAT IT CONTROLS                                                           | DEFAULT           | BUSINESS IMPLICATION                                                                   |
|----------------------------------------------|--------|----------------------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------|
| <code>face_match_threshold</code>            | Number | How close a face match must be to count                                    | 0.4               | Raising it rejects more genuine beneficiaries; lowering it accepts more wrong matches. |
| <code>face_liveness_min</code>               | Number | The minimum liveness score accepted                                        | 0.5               | Guards against a photograph being held up to the camera.                               |
| <code>face_dedup_window_hours</code>         | Number | The window within which the same face at multiple Food Partners is flagged | 6                 | Drives the "face hash repeat" flag in Section 3.10.                                    |
| <code>fraud_anomaly_median_multiple</code>   | Number | How far above the median activity must sit before it is anomalous          | 3                 | Lowering it produces more flags, including more false positives.                       |
| <code>fraud_anomaly_min_count</code>         | Number | The minimum number of events before anomaly detection applies              | 3                 | Prevents a single transaction from being called an anomaly.                            |
| <code>courier_batch_min_value</code>         | Number | The minimum value for a courier batch of printed tokens (₹)                | 5000              | Below this, dispatch by courier costs more than it is worth.                           |
| <code>institution_bulk_allocation_max</code> | Number | The largest bulk allocation to one institution                             | NULL — no limit   | Unset: a bulk allocation of any size is permitted.                                     |
| <code>printed_token_area_lock_default</code> | Text   | The default geographic lock applied to printed tokens                      | NULL — no default | Unset: printed tokens take no area lock unless one is chosen at creation.              |

## 10. Frequently Asked Questions

### 10.1 General

**Q1: Can a beneficiary receive a meal without a smartphone?** A: Yes. A beneficiary does not need a smartphone. A token may be presented through a printed QR code or with the assistance of an authorised volunteer. The Food Partner (shown as "Vendor" in the application interface) scans the token and completes the redemption process. The vendor scan page (`/vendor/scan`) also accepts a pasted code as a fallback.

**Q2: Can a beneficiary receive a meal without Aadhaar or formal identification?** A: Yes. Aadhaar is not mandatory for receiving a pApAmA meal. The platform is designed to ensure that lack of formal identification does not automatically prevent a genuine person in need from receiving assistance. The `aadhaar_hash` field exists in the database schema but is never populated by any registration form. Face verification (via on-device-computed embedding, not photograph) is the primary identity method at redemption, and volunteer-assisted registration ensures accessibility.

**Q3: Does pApAmA cook, store or deliver food?** A: No. pApAmA is a meal-enablement platform. Approved Food Partners prepare the food and serve it directly to the beneficiary. pApAmA does not cook, store or deliver food.

**Q4: Who provides the meal?** A: The meal is prepared and served by an approved pApAmA Food Partner, such as a restaurant, food shop or other authorised meal provider.

**Q5: Can a beneficiary use a token at any restaurant?** A: No. Tokens can only be redeemed at approved pApAmA Food Partners that are active and eligible to participate in the platform.

**Q6: Can a token be exchanged for cash?** A: No. pApAmA tokens are meal-enablement vouchers. They have no cash-withdrawal value and cannot be exchanged for cash, traded, or partially redeemed for change.

**Q7: Can a donor withdraw donated money later?** A: No. Once a donation has been credited as Donor Credit, it represents a commitment to fund meals and is not a withdrawable cash balance. Refunds are only processed for confirmed failed or duplicate payments.

**Q8: What happens when a donor creates a token?** A: The donor's available Donor Credit is reduced by the token amount and a digital meal token is created with a 60-day validity period from activation. The activation date depends on the distribution mode: **Donor Controlled** tokens activate at creation; **PAPAMA Distributed** tokens activate when actually distributed to the beneficiary (confirmed 18 Aug). The donor can either distribute the token personally (Path A — Donor Controlled) or allow pApAmA to distribute it through its authorised volunteer network (Path B — PAPAMA Distributed).

**Q9: Can a donor give a token directly to someone in need?** A: Yes. Under the personal distribution pathway (Path A — Donor Controlled), the donor can share the token QR code directly with a person in need. The token can be displayed digitally or printed. The donor should share the QR discreetly — a live QR is redeemable by whoever presents it first.

**Q10: What happens if a donor chooses "Let pApAmA distribute"?** A: The token enters the Admin Pool and may be allocated to an authorised volunteer, who can then distribute it to an eligible beneficiary.

**Built — September 2026 (work order A-4 and F-5, B-32):** FIFO allocation for pool tokens.

**Q11: What happens if a token is lost?** A: An authorised administrator (or the donor through the donor interface) can report the token as lost. The original token is immediately blocked (`status: "blocked"`) and a replacement is issued with the same value, linked to the original via `replacement_for_token_id`. If the replacement minting fails, the original is automatically unblocked. The action is recorded for audit purposes.

**Q12: What happens if a token expires?** A: Tokens have a **60-day validity period** from activation — activation at creation for Donor Controlled tokens, at distribution to the beneficiary for

PAPAMA Distributed tokens (confirmed 18 Aug). After expiry, the token is **permanently non-redeemable** (token revalidation retirement confirmed 18 Aug); only authorised reissue is permitted. **Approved policy:** Expired token value returns to the Meal Pool for future meals via the controlled reissue process — it is never treated as revenue. An authorised administrator may reissue an expired token, creating a new token with a new QR code and new 60-day validity, permanently linked to the original. **Current behaviour:** The expired token's value is returned to the Meal Pool, never written off and never recorded as revenue.

**Built — September 2026 (work order F-4, B-03):** Implementation of the approved Meal Pool return policy for expired and forfeited token value.

**Q13: What happens if the meal costs less than the token value?** A: The difference between the token value and the approved meal value is recorded by the platform. **Approved policy:** Forfeited value returns to the Meal Pool for future meals. For Special Care Tokens, the surplus is credited to the Common Special Care Pool. **Current behaviour:** The difference is posted to the Meal Pool ledger and funds another meal. It is never recorded as revenue.

**Built — September 2026 (work order F-4, B-03):** Forfeited value will return to the Meal Pool instead of the revenue ledger.

**Q14: Is the beneficiary required to contribute towards the meal?** A: Under the approved operating policy, the beneficiary contribution is ₹10 per meal. The ₹10 is a contribution to pApAmA intended for the pApAmA Administration Account to support approved administrative and operational expenses — it is not Food Partner revenue. The contribution may be collected by the Food Partner on behalf of pApAmA as an authorised collection agent and subsequently remitted to the designated Administration Account. Approved humanitarian waivers apply where a beneficiary is unable to contribute — no beneficiary is ever denied food for inability to pay.

**Built — September 2026 (work order F-1, B-01):** Systematic contribution enforcement, waiver recording, remittance tracking and settlement-release gate within the platform. Current implementation: `co_contribution_max` is configurable (set to 10); the vendor scan UI hardcodes a ₹5 limit pending alignment. **Planned (B-10/B-20):** Hard ₹10 ceiling enforcement and UI alignment.

**Q15: Why does pApAmA ask a beneficiary to contribute ₹10?** A: The contribution is intended to encourage participation and dignity while helping support the administrative and operational costs of the pApAmA programme. It is not intended to make the beneficiary responsible for the cost of the meal. The Foundation provides approved exemptions or waivers in situations where the beneficiary is unable to contribute.

**Q16: Does the Food Partner keep the ₹10 contribution?** A: No. Under the Foundation's approved policy, the ₹10 contribution belongs to the pApAmA Administration Account. Where the Food Partner collects it on behalf of pApAmA, the amount must subsequently be remitted and reconciled with the Foundation. The Food Partner receives the full approved meal value separately through the normal settlement process.

**Q17: Does pApAmA guarantee that every token will provide exactly the same meal?** A: The meal must be selected from the Food Partner's approved menu and comply with the applicable pApAmA meal standards. The Foundation's Standard Meal Framework (a Trust policy document in preparation) will define the minimum requirements for meal value, portion, quality and, where applicable, nutritional standards.

**Q18: Can a beneficiary receive more than one meal in a day?** A: The platform applies configurable meal limits (`max_meals_per_day`) and cooldown periods (`meal_cooldown_hours`). These are intended to ensure fair distribution of donor-funded meals. Special Care or emergency provisions may permit different limits where authorised — category-specific cooldowns and Emergency Mode relaxed limits (4 meals/day, 3-hour cooldown) are supported.

**Q19: Can Special Care beneficiaries receive additional assistance?** A: Yes. The approved Special Care programme provides enhanced nutritional support through ₹100 Special Care Tokens.

Donors sponsor these through a dedicated flow. Where the meal costs less than ₹100, the surplus is credited to the Common Special Care Pool for future Special Care purposes. The approved Special Care Category Master includes: Pregnant Women, Postpartum/Lactating Mothers, and Medically Vulnerable/Patients. The Food Partner sees only "SPECIAL CARE TOKEN – ₹100" — never the diagnosis or specific category.

**Partly built — September 2026 (work order A-3, B-28; the category master, the ₹100 token value and the Common Special Care Pool were built; the donor sponsorship flow is not yet in place):** Full Special Care programme implementation including ₹100 token type, category master, Common Special Care Pool ledger and donor sponsorship flow.

**Q20: What happens during a disaster or emergency?** A: pApAmA operates under an authorised Emergency Mode. Meal-frequency and cooldown parameters are relaxed within approved limits (4 meals/day, 3-hour cooldown, 7-day maximum duration with auto-revert). The ₹10 contribution may be waived. Financial records, token records, audit trails and fraud controls remain fully active. Emergency Mode never overrides token validity, Food Partner suspension, food-safety holds or fraud blocks.

**Q21: Can a beneficiary be denied food because they do not have documents?** A: Lack of Aadhaar, a smartphone or formal identification does not by itself prevent a genuine beneficiary from receiving assistance, subject to the Foundation's approved verification and safeguarding procedures. During Emergency Mode, verification requirements may be further relaxed — but core token and Food Partner controls always apply.

**Q22: How does pApAmA prevent the same token from being used twice?** A: Each token has a unique identity and a QR code derived from an HMAC-SHA256 signature with a server-held secret. Its status is checked atomically during redemption. Once successfully redeemed, the token cannot be redeemed again. Additional fraud controls identify suspicious duplicate or abnormal activity.

**Q23: How does pApAmA prevent repeated claims by the same beneficiary?** A: The platform applies face-embedding verification (matching against stored embeddings from prior redemptions), meal cooldowns, daily meal limits and face-hash repeat detection. Additional fraud-monitoring mechanisms identify suspicious patterns for administrative review.

**Q24: Does pApAmA store the beneficiary's photograph?** A: No. The platform stores no photograph of any person in any mode. Identity verification uses only a non-reversible 1024-dimension mathematical representation (face embedding), computed on-device and transmitted to the server. No face image is ever transmitted to or stored on the server. The embedding cannot be reversed to reconstruct a face image. Proof photos (plate images) show the food, not the person, and are stored privately for audit purposes.

**Q25: Can a beneficiary give feedback about the meal?** A: Yes. Beneficiaries can provide a star rating (1–5), a free-text comment and flag serious concerns as a complaint via a checkbox. The complaint status workflow progresses from open → investigating → resolved/dismissed.

**Planned (B-12):** Predefined complaint categories (quality, quantity, hygiene, staff, delay, redemption, other).

**Q26: What happens if a beneficiary has a complaint about a Food Partner?** A: The complaint is recorded and reviewed by the Foundation. Depending on the nature and seriousness of the issue, the Food Partner may receive corrective guidance under the graduated framework: warning → final warning → penalty/enhanced monitoring → suspension. Serious matters such as food-safety hazards, fraud or conduct posing an immediate risk to beneficiaries may justify immediate suspension.

**Q27: How are Food Partners monitored?** A: Food Partners are monitored through beneficiary feedback ratings, complaint rates, surprise inspections (with quality-score penalties on failure), redemption patterns, proof submissions and fraud indicators. The purpose is both accountability and continuous improvement in food quality and service.



**Q28: Does pApAmA reveal beneficiary information publicly?** A: No. Public-facing information is aggregated and does not disclose personal information relating to beneficiaries or donors. The public Transparency Dashboard (</transparency>) shows only aggregate counts — total donations, meals served, active Food Partners and beneficiaries reached.

**Q29: Can anyone donate to pApAmA?** A: Yes. A person can donate through the public donation facility (</donate> or </donate/qr>) without creating a donor account. Such donations are recorded and administered through the Foundation's General Donation Pool (shown as "Guest Pool" in the application interface).

**Q30: What is a public or guest donation?** A: A public donation is a contribution made by a person who chooses to donate without creating a registered donor account. This is particularly useful for people making one-time donations, event-related contributions or donations through a publicly shared QR code. No contact information is currently captured from guest donors.

**Q31: Will a donor receive confirmation of a public donation?** A: Not currently. No contact information (mobile number, email) is captured from guest donors, so acknowledgement and thank-you communications cannot be delivered. The approved policy is that where the donor provides permitted contact information, pApAmA should provide an appropriate acknowledgement and impact notification when the contribution results in a meal being redeemed.

**Planned (B-11):** Optional contact capture on guest donations for receipts and thank-you notifications, subject to notification channel availability.

**Q32: Can pApAmA donations be made for a particular occasion?** A: Occasion-based giving such as birthdays, anniversaries, memorial donations, festivals, monthly giving and emergency campaigns are part of the Foundation's long-term donation model. Emergency campaigns are supported. Other occasion-based giving is planned for a future phase and is not implemented today.

**Q33: Can donors know what impact their donation has created?** A: Yes. Registered donors can track their donation and token activity through the donor interface (</donor/dashboard>, </donor/impact>). The Foundation also provides public aggregate impact information through the Transparency Dashboard.

**Q34: Does pApAmA provide cash assistance to beneficiaries?** A: No. pApAmA is designed to enable access to freshly prepared meals. Tokens are intended for meal redemption and are not cash benefits. Tokens cannot be exchanged for cash, traded, or partially redeemed.

**Q35: What happens if a Food Partner is temporarily closed or cannot serve a meal?** A: The platform supports Food Partner availability controls: open/closed toggle ([is\\_open](#)), temporary closure with a return date ([temporary\\_closure\\_until](#)) and stock-exhausted status. When a Food Partner is unavailable, redemptions are hard-blocked. The beneficiary nearby list shows the current status (open, closed, temporarily closed, out of stock).

**Planned (B-13):** Beneficiary-facing redirection to nearby open Food Partners when capacity is reached or the outlet is unavailable.

**Q36: What happens if there is a technical problem while redeeming a token?** A: Network errors are caught gracefully ("Network error — please try again."). No offline queue or retry mechanism exists. A meal is not recorded as redeemed unless the transaction is confirmed by the platform — partial commits roll back, and the token remains redeemable. The Food Partner should follow the prescribed exception procedure and should not treat an unconfirmed transaction as a completed redemption.

**Q37: How is donor money protected?** A: pApAmA maintains transaction records, token records, redemption records, settlement records, audit logs and fraud-monitoring controls to provide traceability and accountability for donor-funded activity. Audit logs are append-only and immutable — DB triggers block update/delete even for `service_role`. Token QR codes use HMAC-SHA256 signatures. Proof of service is required before any Food Partner payment.

**Q38: Can pApAmA operate in more than one city?** A: Yes. The initial pilot operates within a defined city boundary using the city lock feature (enforced at redemption only). The approved geographic structure supports: Country → State → District → City/Town/Village/Locality → PIN Code, allowing pApAmA to expand in a controlled manner while maintaining location-wise reporting and accountability. Current build uses city string + coordinates.

**Built — September 2026 (work order A-1, B-02):** Full geographic hierarchy with State/District masters, location IDs and PIN validation. **Built (Q-5 / B-15, October 2026):** City lock is enforced at registration as well as at redemption.

**Q39: What is the basic philosophy of pApAmA?** A: pApAmA is designed to enable meals with dignity. It connects donors, beneficiaries, Food Partners and volunteers through a controlled technology platform so that charitable contributions can be converted into freshly prepared meals for people in need. The platform creates an accountable pathway from: Donation → Donor Credit → Meal Token → Distribution → Beneficiary → Food Partner → Meal Served → Proof → Settlement → Donor Impact Notification.

## 10.2 For Donors

**Q1: Can I get my money back after donating?** A: No. Once a donation is successfully received and credited as Donor Credit, it is non-withdrawable and is committed to the pApAmA programme. The treatment of unused, expired or forfeited token value follows the Foundation's approved financial policy: such value returns to the Meal Pool for future meals and is never treated as revenue. Refunds are only processed for confirmed failed or duplicate payments.

**Built — September 2026 (work order F-4, B-03):** Implementation of the approved Meal Pool return policy.

**Q2: What is Donor Credit?** A: Donor Credit is the non-withdrawable balance representing the donor's committed funds within pApAmA. It increases when the donor makes a donation and decreases when the donor mints a token. The donor can see the credit balance and relevant transaction history at </donor/credit>.

**Q3: Does Donor Credit expire?** A: No. Donor Credit does not expire. It remains available to the donor until it is used to mint a token, subject to the applicable platform and Foundation policies. This is Trust policy, unless required by future statutory or regulatory provisions.

**Q4: What is the difference between Path A and Path B?** A: **Path A — Donor Controlled:** the donor personally decides whom to give the token to and shares it directly with the intended beneficiary. The token goes **live** immediately with a QR code. **Path B — PAPAMA Distributed:** the donor entrusts the token to pApAmA, which places it in the Admin Pool for allocation through authorised volunteers. Path B is preferable where the donor does not personally know a beneficiary or wishes pApAmA to identify and assist someone in need.

**Built — September 2026 (work order A-4 and F-5, B-32):** FIFO allocation for pool tokens.

**Q5: Is a Path A token transferable?** A: A donor-distributed token can be presented by whoever possesses the valid QR code, subject to the platform's redemption controls. Donors should therefore not post or publicly circulate a token unless they intentionally want it to be accessible to anyone who obtains it. Share the QR discreetly and only with the intended person.

**Q6: Can I cancel a token after creating it?** A: There is no donor-side cancellation mechanism. Once minted, a token follows its lifecycle (active → redeemed or expired). An administrator can revoke a token that is currently held by a volunteer (**assigned\_to\_volunteer** status), returning it to the Admin Pool for reallocation. The token value is preserved in the system — it is not returned to the donor's credit balance.

**Q7: What happens if my token is lost?** A: You can report a lost token through the donor interface (</donor/tokens>). The system immediately blocks the original token and issues a replacement with the same value. The original and replacement tokens are permanently linked

via `replacement_for_token_id` for audit purposes. If the replacement minting fails, the original is automatically un-blocked. You can also ask an administrator to report the loss.

**Q8: What happens when my token is redeemed?** A: You receive an in-app notification confirming that the token has been redeemed and that a meal has been served. Per the approved privacy policy, the notification contains: the type of token, the redemption location at City and State level (e.g. "Mumbai, Maharashtra"), and a thank-you message. **Current behaviour:** The notification carries the permitted fields only - token type and value, date and time, the Food Partner's city and state, the Emergency ID where one applies, and the programme. The beneficiary's category is structurally unable to reach the message: the rule is asserted on the payload itself, not on the wording of a template.

**Built — September 2026 (work order P-1, P-2 and P-3, B-29):** Notification whitelist filter and template alignment with approved content.

**Q9: Will I know who received my meal?** A: Per the approved Beneficiary Privacy and Donor Communication policy, donors receive appropriate impact information without exposure to the beneficiary's personal information. You do not see the beneficiary's name, photograph, health status, Special Care category, address or any identifying information. The identity and privacy of beneficiaries is protected. Donor reporting uses anonymised or aggregated information.

**Q10: What happens if my token expires without being used?** A: Tokens have a **60-day validity period** from activation — activation at creation for Donor Controlled tokens, at distribution to the beneficiary for PAPAMA Distributed tokens (confirmed 18 Aug). After expiry, the token is permanently non-redeemable (revalidation retired — confirmed 18 Aug); only authorised reissue is permitted. **Approved policy:** Expired token value returns to the Meal Pool for future meals via the controlled reissue process. An authorised administrator may reissue the token — creating a new token with a new QR code and new 60-day validity, permanently linked to the original. The original remains expired. **Current behaviour:** The expired token's value is returned to the Meal Pool, never written off.

**Built — September 2026 (work order F-4, B-03):** Meal Pool return implementation.  
**Planned (B-23):** Full controlled reissue model including per-mode activation-date rule.

**Q11: What happens if the meal costs less than my token value?** A: The difference is recorded by the platform. **Approved policy:** Forfeited value returns to the Meal Pool for future meals (for Special Care Tokens, to the Common Special Care Pool). This treatment is transparent to donors and consistent in accounting and reporting. **Current behaviour:** The difference is posted to the Meal Pool ledger and funds another meal. It is never recorded as revenue.

**Built — September 2026 (work order F-4, B-03):** Meal Pool return implementation.

**Q12: Can I choose the value of my token?** A: A donation creates Donor Credit. When sufficient credit is available (at least `standard_token_value`), the donor mints a token from the Tokens page (`/donor/tokens`). The token amount must be at least the standard token value and cannot exceed the available credit balance. The standard token value is configured by the Foundation.

**Q13: Can I donate without creating an account?** A: Yes. The Public Donation facility (`/donate` or `/donate/qr`) allows a person to make a donation without registering as a donor. No contact information is currently captured from guest donors.

**Planned (B-11):** Optional contact capture (mobile/email) on guest donations for acknowledgement and thank-you communications.

**Q14: Will a public donor receive a meal-redemption notification?** A: Not currently. No contact information is captured from guest donors, so notifications cannot be delivered. Where a donor provides valid contact information and the applicable notification channel is enabled, the donor should receive an appropriate notification when the contribution results in a meal being redeemed. This is an important part of the donor-impact journey.

**Planned (B-11):** Guest contact capture and notification delivery.

**Q15: Can I make a donation for a particular occasion?** A: Emergency relief campaigns are supported through Emergency Mode. Other occasion-based giving — monthly giving, birthday sponsorships, anniversary donations, memorial donations, festival campaigns — is part of the Foundation's long-term model and is planned for a future phase. It is not implemented today.

**Q16: Can I donate through a birthday, wedding or other event instead of receiving gifts?** A: The long-term model envisions allowing a celebrant to request that guests donate to pApAmA instead of giving gifts, with a dedicated QR code or campaign identifier. This functionality is planned for a future phase and is not implemented today.

**Q17: Can I see the impact of all my donations?** A: Yes. Registered donors can view their donation history, Donor Credit balance, tokens created and meals served through the donor interface (</donor/dashboard>, </donor/impact>). The public Transparency Dashboard (</transparency>) additionally provides aggregate platform-level impact without revealing personal information.

**Q18: Is my donation used only for food?** A: Donor funds credited as Donor Credit are committed to minting meal tokens. The ₹10 beneficiary contribution supports the Foundation's approved administrative and operational expenses and is accounted for separately. Expired or forfeited token value is, under the approved policy, returned to the Meal Pool for future meals rather than being treated as revenue. The Foundation maintains transparent records of all fund flows.

**Built — September 2026 (work order F-4, B-03):** Meal Pool return implementation for expired and forfeited value.

**Q19: Can I donate again after my token has been redeemed?** A: Yes. A donor may make additional donations at any time. The donor's impact history continues to accumulate over time.

**Q20: What is the main difference between donating and sponsoring a meal?** A: A donation places funds into the pApAmA system as Donor Credit, while token creation converts the donor's available credit into a defined meal entitlement (a token) that can subsequently be distributed and redeemed for a freshly prepared meal. The complete journey is: Donation → Donor Credit → Mint Token → Distribute → Meal Redeemed → Donor Notified → Impact Tracked.

### 10.3 For Food Partners

**Q1: Why is my payment locked?** A: Payment remains locked until the required proof of service is submitted and the applicable approval process is completed. This protects both the Food Partner (shown as "Vendor" in the application interface) and pApAmA by ensuring that payments are supported by evidence of the meal served. The payment is not permanently withheld; it moves through the defined approval and settlement process.

**Q2: What proof do I need to submit?** A: The required proof includes a plate/meal photograph showing the food served. The photograph should be clear and sufficiently legible for verification. Photos must show the meal, not the beneficiary. Proof photos are write-once — once uploaded, they cannot be altered or deleted.

**Q3: What if my proof is rejected?** A: You will see the rejection reason on your Redemptions page — proof rejection is the only action in the platform that enforces a mandatory reason at the server level. Fix the issue (e.g. take a clearer photo) and re-upload. There is no deadline for resubmission. Payment stays locked until proof is approved.

**Q4: Why does pApAmA require proof of service?** A: Proof protects all parties: donor funds are protected; Food Partners have evidence supporting their payment claim; beneficiary meals can be verified; fraudulent or duplicate claims can be identified; the Foundation maintains an auditable record of every meal served.

**Q5: How often will I be paid?** A: Settlement cycles may be daily, twice weekly or weekly, depending on the Foundation's configured operating policy. You can view the status of your

settlements at `/vendor/settlements`. The Foundation's operating target is Food Partner payment within 7 working days of settlement reconciliation.

**Q6: What do the settlement statuses mean?** A: **Pending** — approved meals awaiting settlement. **Locked** — settlement prepared and locked for review. **Approved** — reviewed and approved by the checker. **Reconciled** — pre-payment reconciliation complete. **Paid** — payment transferred to the Food Partner. A settlement can be placed **on hold** at any stage before payment for further review.

**Built — September 2026 (work order F-2, B-24):** System-enforced maker-checker — the person who prepares (locks) a settlement cannot be the same person who approves it.

**Q7: Does the Food Partner receive the full approved meal value?** A: Yes. Under the approved Foundation policy, the Food Partner receives the full approved meal value (`min(token_value, menu_value)`) through the normal settlement process. The beneficiary's ₹10 contribution is treated separately and is intended for the pApAmA Administration Account. Where the Food Partner collects the ₹10 on behalf of pApAmA, the amount must be remitted to the designated Administration Account and reconciled separately. These are completely separate financial transactions.

**Q8: Why does the beneficiary pay ₹10?** A: The ₹10 contribution is intended to support the pApAmA Administration and operational account. It is not intended to reduce the Food Partner's approved meal payment. The contribution is a separate financial transaction from the Food Partner's meal settlement.

**Q9: What happens if the beneficiary cannot pay the ₹10?** A: The contribution is waived under the Foundation's approved humanitarian waiver policy. A genuine beneficiary is never denied food solely because they are unable to make the contribution. The waiver is recorded against the relevant redemption transaction and included in settlement and emergency reconciliation. The Food Partner continues to receive the full approved meal value via settlement.

**Built — September 2026 (work order F-1, B-01):** Systematic waiver recording, contribution reconciliation, remittance tracking and settlement-release gate within the platform.

**Q10: Can I charge the beneficiary more than ₹10?** A: No. The Food Partner must not impose any additional charge on a beneficiary beyond the amount authorised by pApAmA. The beneficiary receives the approved meal without being pressured to make any additional payment.

**Q11: Can I serve any meal I want?** A: No. Only meals/items approved through the pApAmA menu process (`/vendor/menu`) may be offered for token redemption. The Food Partner selects the menu item served from their approved list — the beneficiary does not choose. Menu items must be approved by an administrator or vendor manager before they are available for redemptions.

**Q12: Can I change my menu or prices?** A: Food Partners may propose changes through the menu-management process at `/vendor/menu`. Changes to meal items or prices require administrative review and approval before they can be used for pApAmA redemptions. Note that menu price edits overwrite the previous value — no price history is maintained.

**Q13: What is the pApAmA Standard Meal Framework?** A: The Standard Meal Framework will define the Foundation's minimum expectations regarding meal value, portion, quality and, where applicable, nutritional standards. This is a Trust policy document in preparation. Food Partners should comply with the approved framework once it is formally adopted.

**Q14: Can I temporarily close my Food Partner outlet?** A: Yes. Set a `temporary_closure_until` date/time via your availability page (`/vendor/availability`). Redemptions are hard-blocked until the specified time passes. The beneficiary nearby list shows "Temporarily closed".

**Q15: What happens if I reach my daily meal capacity?** A: If capacity enforcement is enabled (`vendor_capacity_enforcement_enabled` = ON) and you reach your configured `daily_meal_capacity`, further redemptions are hard-blocked until the limit resets. This

protects meal quality and prevents Food Partners from accepting more meals than they can properly serve.

**Planned (B-13):** Beneficiary-facing redirection to nearby open Food Partners when capacity is reached.

**Q16: Can I refuse a valid pApAmA token?** A: A Food Partner should not refuse a valid token without a legitimate operational reason. Where the token is invalid, expired, already redeemed or fails an applicable platform rule, the Food Partner should follow the system's prescribed process rather than attempting to bypass it. All token holders are served with equal dignity.

**Q17: Can the same token be redeemed twice?** A: No. Once a token has been successfully redeemed, it cannot be redeemed again. Duplicate scan attempts are detected and blocked. Any suspicious duplicate activity should be reported through the appropriate platform process.

**Q18: What happens if there is a technical problem during redemption?** A: Network errors are caught gracefully ("Network error — please try again."). No offline queue exists. The Food Partner should follow the platform's approved exception procedure and should not treat an unsuccessful or unconfirmed transaction as a completed redemption unless the system or an authorised administrator confirms it. A partially committed redemption rolls back and the token remains redeemable.

**Q19: How are Food Partners monitored?** A: Monitoring includes beneficiary feedback ratings, complaint rates, surprise inspections (failed inspections apply a numeric quality-score deduction via `vendor_inspection_fail_penalty`), redemption patterns, proof submissions and fraud indicators. The purpose is not merely enforcement but also continuous improvement in food quality and service.

**Q20: What happens if I receive complaints?** A: Complaints are investigated fairly. The Foundation follows a graduated corrective-action process: warning → final warning → penalty/enhanced monitoring → suspension. Serious matters such as food-safety hazards, fraud or conduct posing an immediate risk to beneficiaries may justify immediate suspension. The `vendor_auto_suspend_enabled` setting remains OFF — the ladder governs operationally through administrative practice.

**Q21: Can my Food Partner registration be suspended?** A: Yes, where justified under the Foundation's Food Partner governance policy. Suspension may result from serious or repeated quality issues, fraud, food-safety concerns, regulatory non-compliance or other material violations.

**Q22: Can a suspended Food Partner be reinstated?** A: Where the Foundation's policy permits reinstatement, the Food Partner may be reinstated after the identified deficiencies have been corrected and the required review has been completed.

**Q23: What happens if my settlement contains an error?** A: The Food Partner should raise the discrepancy through the prescribed support or settlement-review process. The transaction remains traceable, and any correction is recorded as a separate record rather than altering the historical transaction. Once a settlement is marked as paid, the record is final.

**Planned (B-07):** Auditable settlement adjustment and recovery records.

**Q24: Can pApAmA withhold an entire settlement because of one disputed transaction?** A: The settlement process follows the Foundation's approved financial-control policy. Currently, holds apply to the entire settlement. The approved approach is that only the disputed amount should be placed on hold while undisputed amounts proceed through normal settlement, to protect Food Partner cash flow while maintaining financial controls.

**Planned (B-08):** Line-item settlement hold.

**Q25: Why does pApAmA need photographs of the meal?** A: The meal photograph provides evidence that the claimed meal was actually prepared and served. The purpose is accountability



and fraud prevention, not unnecessary monitoring of Food Partners. Photos show the food, not the person.

**Q26: Can the same photograph be used for another redemption?** A: No. Reusing the same or substantially similar proof photograph for multiple transactions triggers a duplicate-media flag (via perceptual hash comparison at `proof_phash_dup_distance`). Related settlements are automatically placed on hold for review. A flag is not proof of fraud — it is a signal for administrative investigation.

**Q27: Will pApAmA publish my Food Partner information?** A: Only information authorised for operational, transparency or public-facing purposes is published (e.g. name and availability on the beneficiary nearby list). Sensitive KYC documents and financial information are stored in private buckets with role-restricted access and short-lived signed URLs.

**Q28: What is expected from a pApAmA Food Partner?** A: The fundamental responsibility is: serve a safe, hygienic, approved and appropriately portioned meal to the eligible beneficiary with dignity, record the transaction accurately and provide the required evidence for settlement. Collect the ₹10 contribution where applicable, remit it to the pApAmA Administration Account, and cooperate with platform audits and inspections.

## 10.4 For Administrators

**Q1: What must I configure before going live?** A: See the **Go-Live Checklist** section (between Section 2 and Section 3). At minimum: `standard_token_value`, `meal_cooldown_hours`, `max_meals_per_day`, `token_redemption_radius_km`, `max_tokens_per_volunteer`, `token_expiry_days` (set to 60), `co_contribution_max` (set to 10), `settlement_random_audit_rate` (set to 0.10), and the UPI merchant VPA environment variable.

**Q2: What happens if a setting is NULL?** A: Each setting has specific NULL semantics — there is no universal "soft-skip". For example:

- `meal_cooldown_hours` = NULL → no cooldown enforced (unlimited meal frequency)
- `max_meals_per_day` = NULL → no daily limit enforced
- `token_expiry_days` = NULL → tokens never expire (unbounded liability)
- `max_tokens_per_volunteer` = NULL → no holding limit (unbounded exposure)
- `audit_log_retention_days` = NULL → permanent retention (correct default)
- `emergency_meal_cooldown_hours` = NULL → no cooldown during emergency
- `co_contribution_max` = NULL → only ₹0 accepted

See Section 9 for per-key NULL semantics.

**Q3: What is the fail-safe principle?** A: Mandatory configuration settings that are left NULL may allow the system to operate without intended safeguards (e.g. no cooldown, no daily limit, no holding cap). Review and set all mandatory settings before going live.

**Built — September 2026 (work order Q-4, B-06):** Dashboard warning when critical configuration is incomplete.

**Q4: Can I undo a settlement payment?** A: No. Once a settlement is marked as `paid`, the record is final. Corrections to paid settlements are recorded as separate adjustment/recovery records — the original is never edited.

**Planned (B-07):** Auditable settlement adjustment and recovery records.

**Q5: How does maker-checker work for settlements?** A: The person who prepares a settlement should not be the same person who approves and releases payment. This is an operating procedure with audit oversight. The system records the actor for each settlement status transition.

**Built — September 2026 (work order F-2, B-24):** System-enforced maker-checker — maker ≠ checker blocked, settlement versioning, approval auto-invalidation, reject with mandatory reason, bank-account change controls.

**Q6: What settlement pre-payment checks should I perform?** A: Before marking a settlement as paid:

- Verify all included redemptions have approved proofs
- Check that no fraud flags are outstanding for included transactions
- Confirm ₹10 contribution status (collected/waived/outstanding) for each redemption
- Ensure the settlement is not on hold
- The maker-checker principle should be followed as operating practice

**Q7: How do I manage Emergency Mode?** A: Activate at `/admin/emergency`. Set emergency values first: `emergency_max_meals_per_day` = 4, `emergency_meal_cooldown_hours` = 3, `emergency_mode_max_duration_days` = 7. Extension requires explicit action with a revised end date. Emergency Mode auto-reverts after the configured duration. All financial controls remain active.

**Q8: How are configuration changes audited?** A: Every change is logged with the previous value, new value, actor and timestamp. There is no dedicated reason field currently.

**Built — September 2026 (work order Q-3, B-14):** Optional reason field on configuration change audit.



## Appendix A: Food Partner Code of Conduct

---

Food Partners (shown as "Vendor" in the application interface) participating in the pApAmA platform commit to the following standards:

1. **Dignity** — Serve every token holder with the same respect and courtesy as any paying customer. Do not distinguish between token-funded and cash-paying patrons in manner, seating, or speed of service.
  2. **Non-discrimination** — Serve all token holders regardless of category, appearance, gender, religion, caste, or any other characteristic. The platform does not display beneficiary categories to the Food Partner at the point of service.
  3. **Freshness** — All meals served under pApAmA tokens must be freshly prepared. Pre-packaged, reheated or stale food is not acceptable.
  4. **Approved pricing** — Charge only the prices listed in the approved menu. Do not add surcharges, service charges, or other fees beyond the ₹10 beneficiary contribution (collected as an authorised pApAmA collection agent).
  5. **₹10 contribution** — Collect the ₹10 beneficiary contribution only as an authorised collection agent of pApAmA. Remit collections to the pApAmA Administration Account under the CA-approved process. If a beneficiary is unable to pay, waive the contribution — no beneficiary is ever denied food for inability to pay.
  6. **Hygiene** — Maintain food safety and hygiene standards consistent with FSSAI requirements.
  7. **Audit cooperation** — Cooperate with platform audits, including settlement audits, inspections and proof reviews. Upload clear, honest proof photos after every meal served.
  8. **Privacy** — Do not photograph beneficiaries. Do not share any beneficiary information observed during the redemption process. Proof photos must show the meal, not the person. Face images are never retained — only embeddings are computed on-device.
-

## Appendix B: Volunteer Code of Conduct

---

Volunteers distributing tokens on behalf of pApAmA commit to the following standards:

1. **Dignity** — Treat every beneficiary with respect. Distribution is a service, not a favour.
  2. **Non-discrimination** — Distribute tokens equitably. Do not prioritise or exclude beneficiaries based on gender, religion, caste, appearance, or any other characteristic.
  3. **No payment demands** — Never demand money, favours, or anything of value in exchange for a token. Tokens are funded by donors and are free to the beneficiary.
  4. **No token misuse** — Do not redeem tokens for personal use, sell tokens, or divert them from their intended purpose.
  5. **Privacy** — Do not photograph beneficiaries during distribution. Do not share beneficiary personal information. Do not publicise the identity of people receiving tokens.
  6. **Ethical representation** — Represent pApAmA and its mission honestly. Do not make claims about the platform, the Trust, or the donor that are not authorised.
  7. **Facilitate, do not create entitlement** — The volunteer's role is to facilitate access to pApAmA assistance, not to create or alter entitlement. Volunteers shall not independently authorise, modify or override beneficiary eligibility or token parameters.
-

## Appendix C: Business Rules — Data Integrity and Retention

---

**Record preservation:** No donation, token or settlement record is deleted in normal platform operation. All state changes are tracked through status transitions (e.g. a token moves from `live` to `redeemed`, never deleted).

**Compensating rollbacks:** When a transaction fails partway through (e.g. settlement line-item insertion fails after the header is created), the platform performs a compensating cleanup that may delete the orphaned record. These rollbacks are audit-logged. The lost-token replacement flow includes a compensating rollback — if the replacement token minting fails, the original token is automatically un-blocked to restore the original state.

**Audit log immutability:** Audit logs are append-only at the database level. Two PostgreSQL triggers block any update or delete operation, including by `service_role`. No route, service or administrative action can edit or delete an audit log entry.

**Retention:** The `audit_log_retention_days` configuration key exists but is intentionally unset (NULL). No automated purge or retention job runs. Data accumulates indefinitely. Financial, token, settlement and governance records are never deleted. This is the approved retention policy — the key should remain unset until a retention policy is formally adopted with appropriate legal and accounting advice.